Fix Pattern-Aware Vulnerability Patch Generation via In-Context Learning

MIAOMIAO SHAO, Harbin Institute of Technology, Shenzhen, China
YUXIN DING*, Harbin Institute of Technology, Shenzhen, China
CUIYUN GAO*, Harbin Institute of Technology, Shenzhen, China
JUNRU WANG, Harbin Institute of Technology, Shenzhen, China
GUOQING ZHU, Harbin Institute of Technology, Shenzhen, China

With the rapid advancement of deep learning technologies, numerous large language models (LLMs) have been developed, achieving remarkable performance across various tasks. Nevertheless, in the field of vulnerability patch generation, previous studies still face notable limitations: (1) Insufficient context. LLM-based approaches often rely on a single vulnerable function as prompt input for generating patches, which prevents the model from utilizing adequate contextual information and limits its ability to produce accurate patches. (2) Difficulties in extracting effective fix patterns. The substantial semantic differences between various vulnerabilities result in a wide range of fix patterns. Consequently, automatically extracting effective fix patterns tailored to specific vulnerabilities becomes a significant challenge.

To address this, we propose PailGen, a novel fix Pattern-aware, in-context learning-incorporated vulnerability patch Generation approach. PailGen comprises three main modules: hybrid patch retrieval module, fix pattern mining module and in-context learning-incorporated patch generation module. (1) Hybrid patch retrieval module, aims at retrieving the top-*k* relevant vulnerability-fix pairs from the existing codebase by combining BM25 and Dense Passage Retriever (DPR) to construct a hybrid patch retriever. (2) Fix pattern mining module, aims at extracting effective fix patterns from the relevant vulnerability-fix pairs by leveraging an automatic fix pattern mining strategy based on Abstract Syntax Tree (AST). Fix patterns are designed to analyze code changes at different levels (e.g., expression level and statement level). (3) In-context learning-incorporated patch generation module, aims at providing comprehensive contextual information for LLMs by integrating the vulnerable code, its AST, the relevant patches and the extracted fix patterns into the prompt input. Overall, PailGen delivers better demonstrations for in-context learning and provides more comprehensive semantic guidance for patch generation, enabling LLMs to effectively address vulnerabilities. We evaluate PailGen on two real-world C/C++ vulnerability datasets. The experimental results demonstrate that PailGen achieves the best performance, outperforming all baseline approaches.

CCS Concepts: • **Software and its engineering**; • **Security and privacy → Software and application security**;

Additional Key Words and Phrases: Vulnerability patch generation, Retrieval-augmented generation, Fix pattern, Large language models, In-context learning

*Corresponding author

Authors' Contact Information: MIAOMIAO SHAO, 21B951007@stu.hit.edu.cn, Harbin Institute of Technology, Shenzhen, China; YUXIN DING, Harbin Institute of Technology, Shenzhen, China, yxding@hit.edu.cn; CUIYUN GAO, Harbin Institute of Technology, Shenzhen, China, gaocuiyun@hit.edu.cn; JUNRU WANG, Harbin Institute of Technology, Shenzhen, China, 23s151064@stu.hit.edu.cn; GUOQING ZHU, Harbin Institute of Technology, Shenzhen, China, guoqingzhu1996@gmail.com.

ACM 1557-7392/2026/2-ART
https://doi.org/10.1145/3796512

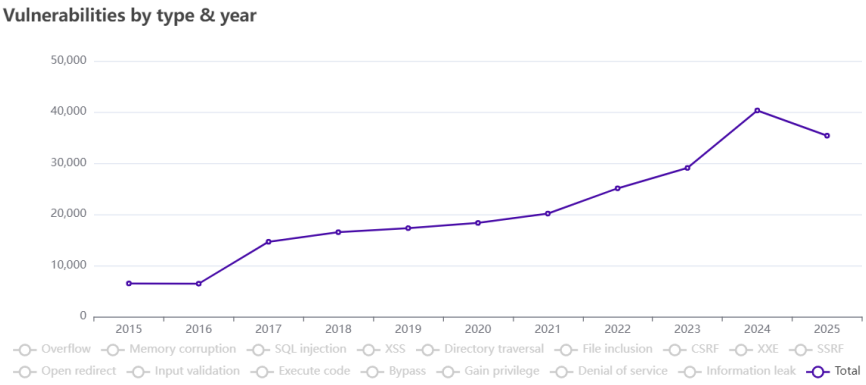


Fig. 1. Annual growth of vulnerabilities in the CVE database [11].

1 Introduction

The rise of open-source software has led to a widespread increase in software vulnerabilities. According to data from the CVE security vulnerability database, the number of newly emerged vulnerabilities in 2024 reached 40,303, reflecting a 38.67% increase compared to 2023 [11], as shown in Figure 1. This trend highlights that threats from vulnerabilities are becoming one of the most concerning security issues. To solve this problem, researchers have developed many vulnerability detection methods. Once a software vulnerability is detected, promptly generating patches for it becomes more important. Manually generating patches is exceedingly time-consuming. Therefore, there is an urgent need for automatic vulnerability patch generation techniques.

A traditional class of techniques for automatic vulnerability patch generation is known as program analysis-based approaches[21, 28, 35, 37]. These approaches often rely on pre-defined rules or patterns to identify and fix vulnerabilities in source code. However, designing rules demands much time and effort from security developers, and pre-defined rules may only be suitable for specific vulnerability types. In recent years, with the rapid advancement of artificial intelligence technology, many deep learning-based vulnerability patch generation approaches have been proposed [7, 8, 19, 73]. These approaches typically formalize vulnerability patch generation as a Neural Machine Translation (NMT) problem, transforming a vulnerable code snippet into a correct version. This translation task is usually achieved by fine-tuning pre-trained models, such as CodeBERT [15], CodeT5 [66]. More recently, large language models (LLMs) have been increasingly applied to various downstream software tasks due to their strong generalization capabilities [23, 30]. In vulnerability patch generation, LLMs can automatically produce patches by taking a vulnerable code snippet and a specific prompt requesting a fix as input. Islam et al. [30] proposed an LLM-based vulnerability patch generation system, SecRepair. This approach employs CodeGen2 [46] to generate fixed code. However, existing state-of-the-art vulnerability patch generation approaches still encounter the following challenges.

Insufficient context. LLM-based approaches often rely on a single vulnerable function as prompt input for generating patches [36, 41]. Due to insufficient context, LLMs often fail on real-world vulnerabilities. Real-world vulnerabilities frequently involve many discontinuous code statements [47], thus providing only a single vulnerable function could make vulnerability analysis difficult. To address this limitation, we find some relevant vulnerability-fix cases from an existing codebase to enrich the contextual information available to LLMs. Specifically, we introduce a patch retrieval module that additionally leverages vulnerability-fix cases retrieved from the codebase to supplement the vulnerability fix knowledge for LLMs. This retrieval strategy is inspired by the practices of software security developers, who often search for relevant vulnerability-fix examples to extract

critical clues for fixing vulnerabilities. We can find some relevant fix cases from the codebase, where the vulnerable code typically exhibits a fix pattern similar to that of the testing vulnerable code. By effectively leveraging these relevant cases, patch generation models can gain a broader context of known solutions, enhancing their ability to generate accurate patches.

Difficulties in extracting effective fix patterns. Some approaches generate fixed code by extracting fix patterns from a set of pre-defined fix templates [62, 69]. A fix template is a pre-defined rule for code transformation that captures common patterns in the vulnerability-fixing process. These templates formalize vulnerability patch generation as a fill-in-the-blank problem by masking multiple tokens related to vulnerabilities in the code. However, such templates are manually defined and may only be suitable for specific scenarios, making it difficult to extract effective fix patterns for various vulnerabilities. More importantly, significant semantic differences exist between different types of vulnerabilities, and even code examples within the same vulnerability type often exhibit substantial variations. This results in a wide range of fix patterns for different vulnerabilities, making it challenging to automatically extract effective fix patterns tailored to specific vulnerabilities. To address this challenge, we automatically extract fix patterns from the retrieved relevant vulnerability-fix pairs. These pairs often share similar fix semantics with the target vulnerable code, enabling the extraction of more precise and targeted fix patterns tailored to specific vulnerabilities. Compared to existing approaches, our approach provides a more fine-grained fix representation. Additionally, by leveraging these fix patterns, we can adaptively and accurately capture the vulnerability-fix semantics corresponding to specific vulnerabilities, enabling more precise and effective patch generation.

In this paper, we propose PailGen, a novel fix Pattern-aware, in-context learning-incorporated vulnerability patch Generation approach. PailGen has three main modules: the hybrid patch retrieval module, the fix pattern mining module and the in-context learning-incorporated patch generation module. The hybrid patch retrieval module utilizes a hybrid patch retriever to automatically retrieve the top-k relevant vulnerability-fix pairs from the existing codebase. The fix pattern mining module extracts specific fix patterns from the relevant vulnerability-fix pairs using AST. Fix patterns are designed to analyze code changes at different levels (e.g., expression level and statement level). Compared to complete vulnerability-fix pairs, the fix pattern extracted from them provides a more fine-grained fix representation. In the in-context learning-incorporated patch generation module, to provide sufficient contextual information for LLMs, we integrate the vulnerable code, its AST, the relevant patches and extracted fix patterns into the prompt input of the LLM. Our approach leverages rich contextual information to improve prompts and guide the in-context learning and inference process of LLMs, eliminating the need for large amounts of annotated data. The source code and datasets presented in this paper are available online[1].

This paper aims to improve the accuracy and practicality of LLM-based vulnerability patch generation methods. To address the limitations of existing mainstream techniques in case retrieval, code representation, and in-context learning, we propose three core innovations. First, we introduce a hybrid retrieval mechanism combining semantic and lexical features. Existing code retrieval methods (e.g., Faiss [12], ColBERT [34]) largely rely on pure semantic matching, which lacks precise alignment with critical code elements (such as dangerous APIs or specific variables) in vulnerability patch generation scenarios. To overcome this, we design a hybrid retrieval framework that integrates semantic methods based on Dense Passage Retrieval (DPR) [32] with lexical methods based on BM25 [56]. BM25 ensures precise matching of API names and code patterns, while DPR captures overall semantic meaning. Furthermore, to better leverage code's structural characteristics, we replace the generic encoder (BERT) in the DPR framework with CodeT5, a model specifically designed for code, and perform targeted fine-tuning. Second, to address the limitation of existing methods that merely concatenate AST embeddings without fully exploiting structural information, we propose a dual-modal AST processing mechanism. For known vulnerability-fix pairs, we extract the core Minimum Edit Tree to precisely represent the fix operations. For target vulnerable

[1]https://github.com/VulDet/PailGen

code that requires fixing, we use its full AST to preserve the complete structural context. This differentiated design enables the model to learn precise modification patterns from retrieved examples while simultaneously generating patches based on the complete structural information of the target code, thereby achieving efficient transfer and application of repair knowledge. Third, we propose a fix rule-guided in-context learning paradigm. Unlike traditional methods that rely on long code pairs for implicit pattern inference, we systematically extract explicit structured fix rules (i.e., fix patterns) from vulnerability-fix cases and inject them as core context into the model. By shifting the focus from "showing code examples" to "teaching fix rules", this approach enhances guidance accuracy and efficiency, improving the model's generalization to unseen or complex vulnerabilities. In summary, this paper makes the following contributions.

- We propose PailGen, an LLM-based automatic vulnerability patch generation framework that combines retrieval-augmented fix pattern mining with in-context learning. To the best of our knowledge, this is the first approach to leverage the power of retrieval in fix pattern mining for LLM-based vulnerability patch generation.
- We propose a novel fix pattern mining scheme that can analyze code changes at different levels. Additionally, we improve LLM prompts by incorporating multiple sources of expert knowledge, including AST of the vulnerable code, relevant patches, and fix patterns.
- We evaluate PailGen on two real-world C/C++ vulnerability-fix datasets. The experimental results demonstrate that PailGen achieves the best performance, outperforming all baseline approaches.

Paper organization. Section 2 outlines the problem definition of our approach. Section 3 describes our approach PailGen. The experimental setup is presented in Section 4, followed by the results in Section 5. Section 6 provides additional discussion of our approach, followed by the limitations of PailGen in Section 7 and a review of related work in Section 8. Finally, Section 9 concludes the article.

2 Task Definition

The goal of vulnerability patch generation approaches is to learn how to map vulnerable code to its patched version. Suppose we have a codebase C that contains a large number of vulnerability-fix pairs. Given a vulnerable function X_i, the retriever first identifies the top-k relevant vulnerability-fix pairs $\{(V_1, F_1), \cdots, (V_k, F_k)\}$ from the codebase C. Each retrieved pair (V_j, F_j) is then parsed into a specific fix pattern T_j. Finally, the source code of the vulnerable function X_i, along with its AST A_i, relevant patches $\{D_1, \cdots, D_k\}$ and fix patterns $\{T_1, \cdots, T_k\}$, are combined to form the input prompts for the LLM. The patch Y_i is subsequently generated by calling the LLM's API. We observe that the length of the entire function code is often too long, making it challenging for LLMs to generate long code sequences accurately. Therefore, rather than using the entire fixed code as the ground truth (i.e., patch), we utilize the *diff* between the vulnerable code and the fixed code. For example, as shown in Figure 2, the ground truth of this vulnerability includes lines 11 and 12. Statements beginning with the "-" symbol indicate vulnerable code lines that should be deleted, while those beginning with the "+" symbol represent the fixed code lines that should be added.

3 Approach

3.1 Overview

In this section, we introduce PailGen, a vulnerability patch generation framework that integrates retrieval-augmented fix pattern mining with in-context learning. PailGen is designed to enhance vulnerability patch generation performance by retrieving relevant vulnerability-fix pairs from the codebase C and extracting fix patterns from them. As illustrated in Figure 3, PailGen consists of three modules: hybrid patch retrieval, fix pattern mining, and in-context learning-incorporated patch generation.

```
1  static int setup_dev_console(const struct lxc_rootfs *rootfs,
2                               const struct lxc_console *console)
3  {
4          char path[MAXPATHLEN];
5          ... /* 19 code lines are skipped for simplicity */
6          if (chmod(console->name, s.st_mode)) {
7                  SYSERROR("failed to set mode '0%o' to '%s'",
8                           s.st_mode, console->name);
9                  return -1;
10         }
11 -       if (mount(console->name, path, "none", MS_BIND, 0)) {
12 +       if (safe_mount(console->name, path, "none", MS_BIND, 0, rootfs->mount)) {
13                 ERROR("failed to mount '%s' on '%s'", console->name, path);
14                 return -1;
15         }
16         INFO("console has been setup");
17         return 0;
18 }
```

Fig. 2. A vulnerable function and its fixes (CVE-2015-1335) from the Big-Vul dataset.

① **Hybrid Patch Retrieval.** Given a vulnerable function X_i, we retrieve the top-k relevant vulnerability-fix pairs from the codebase C using a hybrid retriever. This hybrid retriever combines BM25 and Dense Passage Retriever (DPR), allowing us to capture both lexical and semantic similarities between functions.

② **Fix Pattern Mining.** This module aims to parse each retrieved vulnerability-fix pair (V_j, F_j) into specific fix pattern T_j. First, we transform both the vulnerable function V_j and the fixed code F_j into ASTs. Next, we extract a Minimum Edit Tree (MET) from the ASTs by utilizing information about the vulnerable and fixed code lines. Finally, we generate a set of edit paths from the MET and convert them into a fix pattern.

③ **In-context Learning-incorporated Patch Generation.** In this module, we aim to improve LLM prompts with domain-specific knowledge by combining the source code and AST of the vulnerable function with relevant patches and fix patterns. This approach enables the generation of multiple, more targeted vulnerability patch generation prompts, thereby improving the in-context learning capabilities of LLMs. Next, we provide a detailed description of each module.

3.2 Hybrid Patch Retrieval

Given a vulnerable function, the hybrid patch retrieval module aims to retrieve vulnerability-fix pairs that are both lexically and semantically similar to the testing vulnerable code. We present a hybrid retriever that calculates the similarity between the vulnerable code (i.e., query code) in the test dataset and the vulnerable code (i.e., key code) in the codebase by combining scores from the sparse retriever BM25 and the dense retriever DPR. The BM25 retriever captures lexical similarities, while the DPR retriever focuses on deeper program semantics.

⑴ⓐ **Lexical-based Sparse Retrieval.** We use BM25 [56] as the sparse retriever. It treats each function as a token sequence and converts it into a bag-of-words representation. It then calculates the lexical similarity between the query code X_i in the test dataset and the key code V_j in the codebase C. The similarity score is represented as BM25(X_i, V_j). However, as a lexical-based retriever, BM25 is sensitive to variations in identifier naming within source code.

⑴ⓑ **Semantic-based Dense Retrieval.** The DPR retriever [32] retrieves relevant vulnerability-fix pairs by measuring the semantic similarity between the query code X_i and the key code V_j. To achieve this, we use two independent Transformer-based encoders, E_C and E_Q. Each code is mapped to a d-dimensional dense vector. Specifically, we initialize the DPR model with the pre-trained CodeT5 encoder [66] and then fine-tune it for the

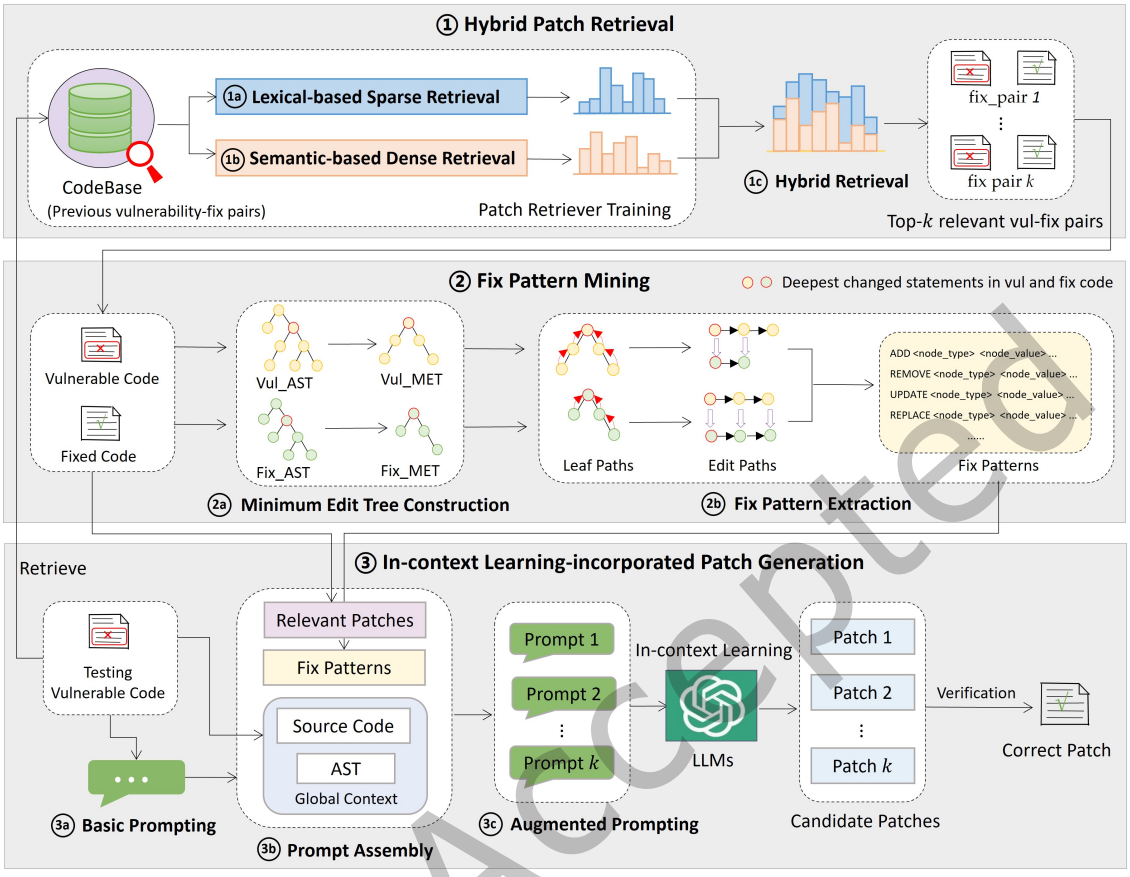


Fig. 3. Overview of PailGen.

code-to-code retrieval task. The encoder E_C encodes the key code V_j and builds an index for it. The query code X_i is encoded by the encoder E_Q. To train the DPR retriever, we use vulnerability-fix pairs from the codebase C as training data. Specifically, in the training process, we use the vulnerable code V_j as the query and the corresponding fixed code F_j as the key. This approach eliminates the need for extensive manual annotation to construct new datasets. For each query code V_j and key code F_j, we use the representation of the `[CLS]` token as the output, and the similarity is calculated as follows:

$$\mathrm{DPR}(V_j, F_j) = E_C(F_j)^{\mathsf{T}} E_Q(V_j). \tag{1}$$

At the training stage, we adopt in-batch negatives to optimize the InfoNCE contrastive loss:

$$L = \frac{1}{|M|} \sum_{j=1}^{|M|} -\log \frac{\exp(\mathrm{DPR}(V_j, F_j))}{\exp(\mathrm{DPR}(V_j, F_j)) + \sum_{l \in M, l \neq j} \exp(\mathrm{DPR}(V_l, F_l))}, \tag{2}$$

where M represents the current minibatch. It contains $|M|$ positive training examples. For each positive example, there are $|M| - 1$ negative examples. The training objective of DPR is to maximize the similarity between positive examples while minimizing the similarity between negative ones.

①c **Hybrid Retrieval.** To incorporate both lexical and semantic information, we follow [32] and use a hybrid retriever that combines BM25 and DPR. The hybrid similarity score is calculated as follows:

$$Hybrid(X_i, V_j) = \text{DPR}(X_i, V_j) + \lambda \text{BM25}(X_i, V_j). \tag{3}$$

Here, the weight parameter λ balances the two retrievers, and is set to 1 in our experiments. Specifically, we retrieve two initial sets of top-k similar vulnerability-fix pairs based on BM25 and DPR, respectively, and re-rank their union using the hybrid similarity score $Hybrid(X_i, V_j)$ as the ranking function. Based on this, we select the top-k relevant vulnerability-fix pairs to guide the fix pattern mining process.

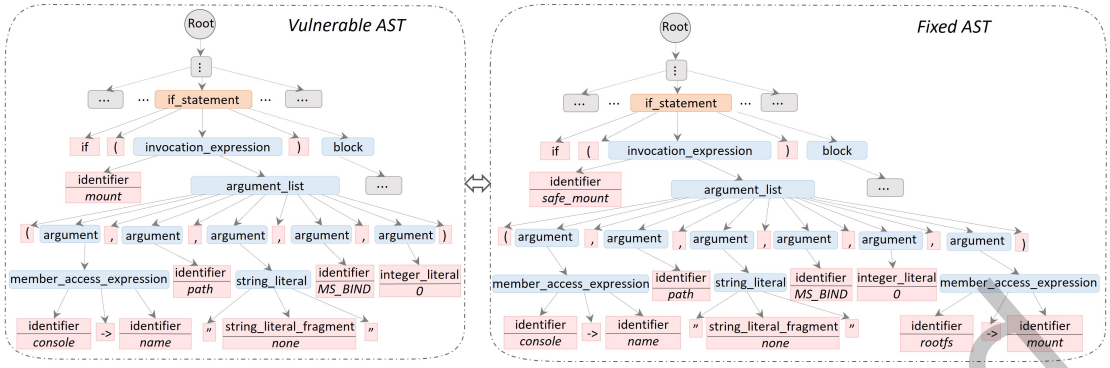
3.3 Fix Pattern Mining

Previous deep learning-based patch generation methods often utilize the complete source code of vulnerable functions as input [8, 18, 19]. However, after analyzing numerous vulnerability-fix cases, we observe that fixes usually involve only a few statements within a function. Using the entire function as input may lead to unnecessary modifications, potentially introducing new issues. Moreover, this function-level, coarse-grained fix representation limits the model's ability to accurately locate core parts of the vulnerability, especially those aspects closely linked to vulnerabilities but not explicitly identified. Additionally, existing methods often treat the vulnerable function as a natural text sequence, neglecting the structural characteristics of code. To address these limitations, we propose a fine-grained, fix pattern-aware fix representation method that enhances both the precision of vulnerability fixes and the capture of the code's structural information.

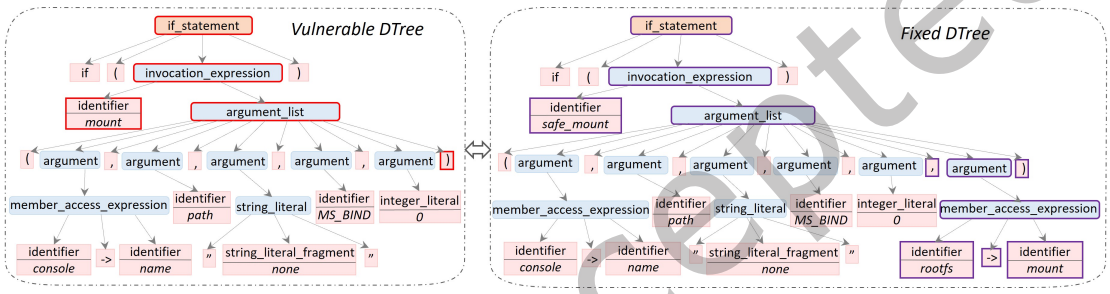
As shown in Figure 3, the fix pattern mining module contains two stages: MET construction and fix pattern extraction. In the MET construction stage, we aim to eliminate code tokens that are not relevant to vulnerabilities by comparing the ASTs of the vulnerable code and the fixed code. This stage allows us to construct a MET that extracts expression-level and statement-level code changes. In the fix pattern extraction stage, a set of edit paths is derived from the MET and then converted into a fix pattern. Fix patterns are designed to analyze code changes at both the expression level and statement level. PailGen parses all relevant vulnerability-fix pairs retrieved from the codebase into specific fix patterns, with an example provided in Figure 4.

②a **Minimum Edit Tree Construction.** Code representation plays a crucial role in analyzing and fixing code vulnerabilities. However, existing vulnerability patch generation approaches typically treat vulnerable code as a text sequence, ignoring the complex structural information inherent in the source code. AST offers a code representation that effectively captures program's structural characteristics. AST is a hierarchical structure that breaks down each code element into smaller components. Thus, we extract fix patterns from AST to retain the structural features of the code.

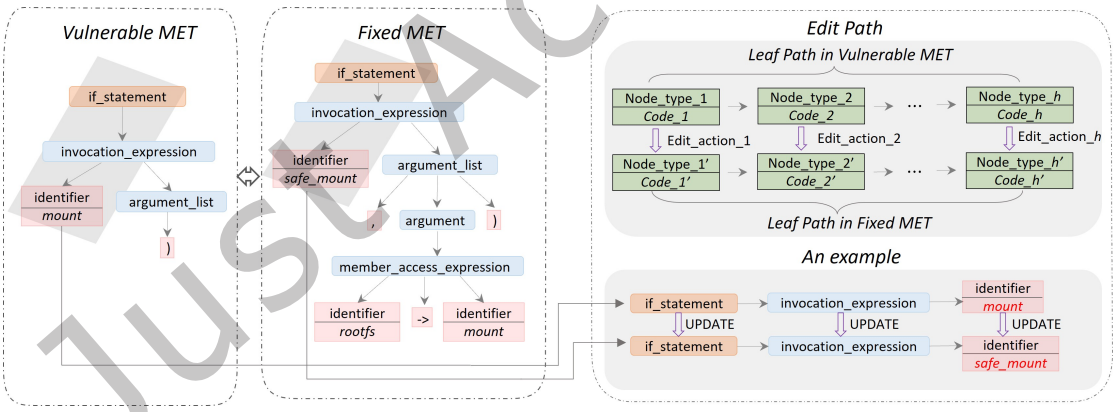
Given the vulnerability-fix pair (V_j, F_j), we first use the tree-sitter tool to parse both the vulnerable code V_j and the fixed code F_j into ASTs, referred to as *Vulnerable AST* (i.e., *Vul_AST*) and *Fixed AST* (i.e., *Fix_AST*), respectively. Figure 4a presents the ASTs of the vulnerable and fixed versions of the function `setup_dev_console` in Figure 2. We then extract subtrees from these ASTs based on the deleted and added code lines, focusing on the deepest changed statements where the code changes occurred. Keeping the original AST node type *t*, we introduce a base type *bt* by reclassifying the original AST node types into two base types: {*Expr*, *Stmt*}. For example, AST node types `expression_statement`, `if_statement`, and `local_declaration_statement` belong to the same base type *Stmt*. We utilize base types to identify the deepest changed statement-level nodes in the AST. The deepest changed statement-level node has a base type of *Stmt*, with code tokens containing one complete vulnerable or fixed statement. This node typically includes at least one *Expr* child node or no *Stmt* child nodes. The subtrees extracted from *Vul_AST* and *Fix_AST* are named *Vulnerable DTree* (i.e., *Vul_DTree*) and *Fixed DTree* (i.e., *Fix_DTree*), respectively. As shown in Figure 4b, PailGen prunes the same subtrees shared by *Vul_AST* and *Fix_AST*, leaving only the modified parts. Specifically, the body of the `if_statement` node (the `block` node) is pruned, leaving only the condition as the deepest changed statement.



(a) Abstract Syntax Trees (ASTs)



(b) Subtrees of the ASTs corresponding to the deepest changed statements



(c) Minimum Edit Trees (METs) and Edit Paths

Fig. 4. The intermediate process of mining fix patterns for the example in Figure 2. (a) For simplicity, irrelevant nodes in the AST are omitted and represented by gray boxes. (b) The red box indicates the modified node in the Vulnerable DTree, while the purple box indicates the modified node in the Fixed DTree. (c) The content enclosed in a gray parallelogram illustrates a specific edit path from a node of type *if_statement* to a leaf node of type *identifier*. The code tokens of the first two nodes are omitted.

Next, we construct Minimum Edit Trees (METs) for *Vul_DTree* and *Fix_DTree*. This step is crucial because vulnerable functions often contain numerous code tokens, but only a small portion is modified during fixing. Constructing the METs allows us to focus on the core parts of the function that requires to be fixed. Additionally, by utilizing the MET, the model's search space for fixing vulnerabilities is significantly reduced. To facilitate understanding, we provide a formal definition of the MET.

Definition 3.1. (Minimum Edit Tree). A Minimum Edit Tree (MET) is a tree (N, E, rt) with nodes N, edges E and root node $rt \in N$. An edge $e \in E$ is a pair (n_p, n_g) where node n_p is the parent of node n_g. A node $n \in N$ (and $n \neq rt$) is represented as a quadruple (bt, t, c, i), where $bt \in \{Expr, Stmt\}$ denotes the base type of node, t is the AST node type, c represents the code tokens of node, and i indicates the node location as the i-th child of its parent node. The root node rt is the deepest statement-level AST node that contains modified code lines. The MET represents the minimum subtree of an AST necessary for transitioning the code from one state to another. The leaf nodes of the MET indicate the smallest edit units in the code.

Definition 3.2. (Leaf Path). Given the Minimum Edit Tree (N, E, rt), the leaf node path $path(ln)$ is an ordered sequence of nodes from the root node rt to the leaf node ln, satisfying the following conditions: 1) the first element of $path(ln)$ is rt; 2) for any consecutive node pair (n_i, n_{i+1}) in $path(ln)$, n_{i+1} is a child of node n_i; 3) the last element of $path(ln)$ is the leaf node ln.

We define the MET to analyze edits at different levels. Specifically, expression-level edits modify individual expressions (i.e., partial tokens) in statements, while statement-level edits involves adding, removing or replacing entire statements. We outline the process of constructing the MET. For *Vul_DTree*, we first identify all leaf nodes. Starting from each leaf node, we traverse the tree from the bottom up to the root node, then backtrack from the root to the leaf node to generate a leaf path. The same procedure is then applied to the *Fix_DTree*. After generating the leaf paths, we compare each leaf path $path(ln)$ in *Vul_DTree* with leaf paths in *Fix_DTree* to determine if there is a matching path. We establish the following rule to check whether a leaf path in *Vul_DTree* matches one in *Fix_DTree*.

Definition 3.3. (Leaf Path Match). Consider a leaf path $path(a) = \{a_1, ..., a_p\}$ in *Vul_DTree* and a leaf path $path(b) = \{b_1, ..., b_q\}$ in *Fix_tree*. Here, p and q represent the lengths of the leaf paths. The nodes a_i and b_i are the i-th nodes on $path(a)$ and $path(b)$, respectively, with a_p and b_q as the respective leaf nodes. The $path(a)$ matches $path(b)$ if both paths have the same length (i.e., $p = q$) and there exists node maps $\{a_1 \rightarrow b_1, \cdots, a_p \rightarrow b_p\}$ where each a_i matches b_i. The nodes a_i and b_i match if their node type, code tokens, and node position are identical. Note that since a_1 and b_1 are root nodes, only their node type and code tokens are compared.

Leveraging the above rule, we identify all matching leaf path pairs between *Vul_DTree* and *Fix_DTree*, then remove the corresponding leaf nodes from both trees. To match the leaf paths, we follow a top-down approach. We start with the root node, and perform type and code token matching between the two nodes before comparing their child nodes. Next, to construct the Minimum Edit Trees, we adopt a bottom-up methodology. Starting from the remaining leaf nodes, we traverse and parse *Vul_DTree* and *Fix_DTree* from the bottom up to the root node, thereby generating the METs *Vul_MET* (i.e., *Vulnerable MET*) and *Fix_MET* (i.e., *Fixed MET*), respectively. Since the *Vul_MET* and *Fix_MET* contain only the modified code lines, we can check whether the edit is at the expression level or the statement level. If *Vul_DTree* and *Fix_DTree* share the same root node, the edit is identified as expression-level, meaning it does not change the entire statement. Otherwise, the edit is considered to occur at the statement level. For example in Figure 4c, the *Vul_MET* and *Fix_MET* have the same root node, so the edits in the `setup_dev_console` function are at the expression level. Note that the *Vul_MET* will be empty if a vulnerability-fix pair includes only added lines and no deleted lines. Conversely, the *Fix_MET* will be empty if a vulnerability-fix pair includes only deleted lines and no added lines.

② Fix Pattern Extraction. After constructing the Minimum Edit Trees, the next step is to investigate how to represent the transition from *Vul_MET* to *Fix_MET*. To achieve this, we design a set of edit paths that illustrate the transition and then mine fix patterns from these paths. The edit path is essentially a sequence of differences between two corresponding syntax paths in the vulnerable code and the fixed code. As shown in Figure 4c, it is constructed by aligning leaf paths in the two METs, where each pair of nodes represents a specific change in node type or code tokens. The construction process is as follows: (1) Path matching: Select semantically corresponding leaf paths from the *Vulnerable MET* and the *Fixed MET*. (2) Differential comparison: Compare the node types and code tokens along these paths, node by node. (3) Path generation: Connect all nodes with differing types or code tokens sequentially to form the final edit path. Mining fix patterns allows us to generalize and extract fix patterns shared by similar code vulnerabilities. We first give formal definitions of edit paths and fix patterns.

Definition 3.4. (Edit Path). An edit path $EP_i(a, b)$ is a map $path(a) \rightarrow path(b)$, where $path(a) = \{v_1, \cdots, v_o\}$ and $path(b) = \{f_1, \cdots, f_m\}$ represent the i-th leaf paths in *Vul_MET* and *Fix_MET*, respectively. The edit path $EP_i(a, b)$ essentially consists of a set of node pairs $\{(v_1, f_1), \cdots, (v_h, f_h)\}$, where $h \leq \min(o, m)$. Each pair (v_i, f_i) indicates the i-th code element and its changes at the corresponding position between *Vul_MET* and *Fix_MET*. Here, h represents the number of nodes along the edit path that involve code changes.

Definition 3.5. (Fix Pattern). A fix pattern T is an ordered sequence that captures multi-level AST node information. It provides the code change patterns for fixing a vulnerable code. These patterns are organized into a set of node edit operations, denoted as S_T. Each element $s \in S_T$ is a quadruple (op, nt, ct, pos) where $op \in \{ADD, DELETE, UPDATE, REMOVE\}$ is the node edit action, *nt* is the AST node type, *ct* is the code tokens of node, and *pos* is the node location. The fix pattern describes the necessary edit operations to transition the vulnerable code to its fixed version.

Node edit actions are classified into four types: *ADD*, *REMOVE*, *UPDATE* and *REPLACE*. The *ADD* action is used when $Vul\_MET = \emptyset$ and $Fix\_MET \neq \emptyset$, while the *REMOVE* action is utilized when $Vul\_MET \neq \emptyset$ and $Fix\_MET = \emptyset$. When both *Vul_MET* and *Fix_MET* are non-empty, we need to select the appropriate edit actions based on the level of the edit. If the edit is at the statement level, we apply *REMOVE* or *ADD* actions to the *Vul_MET* and *Fix_MET*, respectively. Otherwise, we further examine changes in the code tokens and AST node types to determine which edit action to adopt. The descriptions of the fix patterns for these four types of edit actions are as follows:

ADD. The edit action $ADD(n_g(t, c, i), n_p)$ refers to inserting a new node n_g in the original AST, where c represents the code tokens, t denotes the AST node type and i represents the node location. If the parent node n_p is specified, the node n_g is inserted as the i-th child node of n_p; otherwise, n_g becomes the root node.
[rectangle, rounded corners, fill=gray!10, line width=0.5pt, text width=14.9cm]Expression of *ADD* Action:
"ADD" <Node_Type> "@@" <Node_Tokens> "@TO@" <Parent_Node_Type> "@@" <Parent_Node_Tokens> "@AT@" <Node_Location>;

REMOVE. The edit action $REMOVE(n_g(t, c, i))$ refers to removing the leaf node n_g from the AST. The type of node n_g is t, and its code tokens are represented by c.
[rectangle, rounded corners, fill=gray!10, line width=0.5pt, text width=14.9cm]Expression of *REMOVE* Action:
"REMOVE" <Node_Type> "@@" <Node_Tokens> "@AT@" <Node_Location>;

UPDATE. The edit action $UPDATE(n_g(t, c_1, i), n'_g(t, c_2, i))$ involves replacing the code tokens of node n_g. The type of the node n_g is t, with c_1 and c_2 representing the code tokens before and after the change, respectively.
[rectangle, rounded corners, fill=gray!10, line width=0.5pt, text width=14.9cm]Expression of *UPDATE* Action:
"UPDATE" <Node_Type> "@@" <Before_Node_Tokens> "@TO@" <After_Node_Tokens> "@AT@" <Node_Location>;

REPLACE. The edit action $REPLACE(n_g(t, c, i), n_p)$ involves replacing the i-th child of node n_p with node n_g. The type and code tokens of node n_g are t and c, respectively.

```
UPDATE if_statement @@ if (mount(console->name, path, "none", MS_BIND, 0)) { ERROR("failed to mount '%s' on '%s'", console->name, path);
        return   -1; } @TO@ if (safe_mount(console->name, path, "none", MS_BIND, 0, rootfs->mount)) { ERROR("failed to mount '%s' on
        '%s'", console->name, path); return -1; } @AT@ root_node
--- UPDATE invocation_expression @@ mount(console->name, path, "none", MS_BIND, 0) @TO@ safe_mount(console->name, path, "none",
        MS_BIND, 0, rootfs->mount) @AT@ child_2
------ UPDATE identifier @@ mount @TO@ safe_mount @AT@ child_2_0

UPDATE if_statement @@ if (mount(console->name, path, "none", MS_BIND, 0)) { ERROR("failed to mount '%s' on '%s'", console->name, path);
        return -1; } @TO@ if (safe_mount(console->name, path, "none", MS_BIND, 0, rootfs->mount)) { ERROR("failed to mount '%s' on
        '%s'", console->name, path); return -1; } @AT@ root_node
--- UPDATE invocation_expression @@ mount(console->name, path, "none", MS_BIND, 0) @TO@ safe_mount(console->name, path, "none",
        MS_BIND, 0, rootfs->mount) @AT@ child_2
------ UPDATE argument_list @@ (console->name, path, "none", MS_BIND, 0) @TO@ (console->name, path, "none", MS_BIND, 0, rootfs->mount)
        @AT@ child_2_1
--------- REPLACE ,@@, @TO@ if_statement @@ if (safe_mount(console->name, path, "none", MS_BIND, 0, rootfs->mount)) { ERROR("failed to
        mount '%s' on '%s'", console->name, path); return -1; } @AT@ child_2_1_10
.....
```

Fig. 5. Fix pattern for the vulnerable function `setup_dev_console` in Figure 2.

[rectangle, rounded corners, fill=gray!10, line width=0.5pt, text width=14.9cm]Expression of *REPLACE* Action: *"REPLACE" <Node_Type> "@@" <Node_Tokens> "@TO@" <Parent_Node_Type> "@@" <Parent_Node_Tokens> "@AT@" <Node_Location>*;

Figure 5 illustrates the fix pattern extracted from the vulnerability-fix pair in Figure 2. The first line details the code changes made to the root node in the MET, with "*UPDATE*" indicating the type of node edit action. The tokens between "*@@*" and "*@TO@*" represent the code tokens of the node before the change, while the tokens between "*@TO@*" and "*@AT@*" correspond to the new code tokens after the change. The "− − −" symbols represent the level within the AST, signifying that the node on the current line is a child of the node on the previous line, and its own child nodes are represented by "− − − − −−". This notation preserves the hierarchical structure of the nodes in the AST, enabling the model to accurately analyze and fix code vulnerabilities.

3.4 In-context Learning-incorporated Patch Generation

In this module, we select the extracted expert knowledge, including relevant patches and specific fix patterns as the in-context examples for patch generation. Specifically, we introduce a prompt assembly step that enriches the contextual information provided to the LLM by incorporating multiple sources of expert knowledge. Finally, we utilize the contextual information from the prompt for in-context learning and inference, enabling the generation of accurate vulnerability patches.

③a Basic Prompting. To evaluate the impact of incorporating expert knowledge on LLM performance in vulnerability patch generation, we first design a basic prompt to generate patches. We use the following basic prompt and instruct the LLM to provide an explicit response, specifying which statements should be deleted and which should be added.

[rectangle, rounded corners, fill=blue!7, line width=0.5pt, text width=14.9cm, minimum height=1.6cm]"### Vulnerable code: {*code*} ### Task: The code contains a vulnerability. Note that `<S2SV_StartVul>` and `<S2SV_EndVul>` indicate the start and the end of vulnerable code lines. Thus the vulnerable code lines are: {*vul_line*}. Please generate a diff to fix the vulnerability.";

The {*code*} indicates the entire input vulnerable function, while {*vul_line*} refers to the vulnerable code lines within that function.

③b Prompt Assembly. Based on this basic prompt, we introduce an augmented prompt that integrate expert knowledge. For a given vulnerable function, a total of m prompts are generated, each composed of three key components: global context, relevant patches, and fix patterns.

- **Component Related to Global Context.** This component of the prompt includes the entire vulnerable function along with its AST. First, we use a tree-sitter parser to convert the vulnerable function into an AST. Since LLMs work with sequential data, we convert the AST into a node sequence using a depth-first traversal method.
- **Component Related to Relevant Patches.** We incorporate patches from relevant fix cases into the prompt, where a patch refers to the *diff* between the vulnerable code and the fixed code. The *diff* typically has a shorter sequence, enabling LLMs to capture the context of the vulnerable code more accurately while reducing computational costs.
- **Component Related to Fix Patterns.** Compared to patches, the fix pattern offers a more comprehensive fine-grained fix representation. It not only captures the structural characteristics of the vulnerable and fixed code, but also analyzes edits at different levels (e.g., expression-level and statement-level). Thus, we use the fix pattern as an important complement to relevant patches and incorporate it into the LLM prompt.

③c **Augmented Prompting.** PailGen generates final improved prompts by integrating all input components into a single sequence. Finally, we have meticulously assembled the following prompts to guide the LLM in generating patches:

[rectangle, rounded corners, fill=blue!7, line width=0.5pt, text width=14.9cm, minimum height=2.6cm]"### Vulnerable code: {*code*} ### The Abstract Syntax Tree (AST) of the code is: {*ast*}. ### Task: The code contains a vulnerability. Note that `<S2SV_StartVul>` and `<S2SV_EndVul>` indicate the start and the end of vulnerable code lines. Thus the vulnerable code lines are: {*vul_line*}. Please generate a diff to fix the vulnerability. Here is an example of relevant patches: {*relevant_patch*} and the fix pattern extracted from the AST of relevant vulnerability-fix pair: {*fix_pattern*}.";

The {*ast*}, {*relevant_patch*}, and {*fix_pattern*} are filled with the AST of the vulnerable function, the relevant patch, and the extracted fix pattern from the relevant vulnerability-fix pair, respectively.

Although the prompts include instructions for output formats, unexpected results may still occur in the LLM's output, such as natural language explanations of how to fix vulnerabilities. Therefore, we need to further extract the patch from the LLM's output. Specifically, we follow three steps to extract patches from the LLM's output.

- Identify code blocks: When generating vulnerability repair solutions, large language models tend to encapsulate key code differences (diffs) within structured code blocks. These blocks are typically delineated by specific markers (such as triple backticks "```") and often preceded by explicit introductory statements, such as "here is the diff". Based on this observation, we employ regular expressions to scan the LLM's complete output and locate these structured code blocks. This approach effectively filters out lengthy explanatory text, allowing us to accurately extract the pure diff content that conveys the comparison between the vulnerable code and its corresponding fix.
- Parse diff format: To precisely extract the target patch from the identified code blocks, we parse their content to determine whether they conform to the standard diff format. First, we detect file headers starting with "- - -" (original file) and "+++" (modified file). We then analyze the code segments marked by "@@ ... @@" and accurately extract all added lines (prefixed with "+") and deleted lines (prefixed with "-"). This process ensures that the core code change logic is captured from the structured diff comparison while automatically filtering out any statements unrelated to the code modifications.
- Output purification and standardization: To obtain clean and directly applicable patched code, we perform a final filtering step. This process removes all non-functional elements, including code comments and redundant symbols introduced during parsing, ensuring that the resulting patch is syntactically complete and clean. In cases where the LLM returns multiple code blocks with correct formatting but differing

Table 1. Statistics on Our Datasets

Dataset	# CWE IDs	# Functions	Codebase	Test
Big-Vul[14]+CVEfixes[4] (Big-Vul_CVEfixes)	115	4930	3944	986
D2A [72]	-	2535	2028	507
Total	115	7465	5972	1493

content, we select the first grammatically complete block. If no suitable block is found, the sample is considered a repair failure.

Through these steps, we effectively manage the uncertainty of LLM outputs, reliably converting unstructured text responses into structured code changes, thereby ensuring the stability and accuracy of the patch generation process.

4 Experimental Design

4.1 Datasets

To evaluate the effectiveness of PailGen in fixing C/C++ code vulnerabilities, we use the same datasets (i.e., Big-Vul and CVEfixes) that are popularly utilized to evaluate existing vulnerability patch generation approaches [7, 19, 73]. Big-Vul [14] was created by crawling the Common Vulnerabilities and Exposures (CVE) database to extract vulnerability-related data, such as CWE IDs. CVEfixes [4] was collected from the National Vulnerability Database (NVD). Moreover, to assess the applicability of our PailGen for fixing vulnerabilities in the wild, we use the D2A dataset [72]. D2A was constructed by analyzing six large open-source software projects from GitHub. For each project, the authors selected relevant commits that fixed vulnerabilities and ran static analysis on the versions before and after these commits.

By combining the datasets mentioned above, we collected a total of 10,810 vulnerability-fix pairs. However, merging the Big-Vul and CVEfixes datasets may introduce duplicate data, potentially leading to label leakage issues and affecting the fairness of model evaluation. After inspection, we observed that approximately 24% of the pairs contained duplicate vulnerable or fixed functions. To address this, we performed a deduplication process on the original dataset. Each sample in our dataset consists of a vulnerable code and its corresponding patched code. Therefore, if the vulnerable code in one sample is the same as the vulnerable code in another sample, we consider it a duplicate and remove it. Similarly, if the patched code in one sample is identical to that in another, it is also removed from the dataset. Next, we utilized special tokens `<S2SV_StartVul>` and `<S2SV_EndVul>` to mark the beginning and end of vulnerable code lines. The ground truth is the *diff* version between the vulnerable and fixed code. Additionally, we followed the methodology in [7] and limited the input length of vulnerable functions to 1000 tokens and the ground truth to 100 tokens to manage the model's memory requirements.

After data preprocessing, our dataset contains 7465 function-level vulnerability-fix pairs, covering 115 vulnerability types (i.e., CWE IDs). We followed the previous vulnerability patch generation approaches [19, 73], allocating 20% of the dataset for testing. Then we used the remaining 80% of the dataset to build the codebase and train our retrieval module. Table 1 presents the number of vulnerability types (i.e., CWE IDs), the number of function-level vulnerability-fix pairs, and the distribution of samples between the codebase and test set in our dataset.

Additionally, note that both the Big-Vul_CVEfixes and D2A datasets are randomly split into training and testing sets. As a result, future data may be used for training while past data is tested, potentially leading to data contamination. Moreover, these datasets may already be included in the training set of the large model,

which could further introduce data leakage problem. To ensure a fairer evaluation of PailGen, we collected a new vulnerability dataset CVED, which contains 1550 samples. The specific collection process for CVED is as follows:

We collected open-source vulnerabilities from the CVE security vulnerability database[2]. During the collection process, we retrieved CVE entries in chronological order and stored them in a structured format. Each entry includes key features such as CVE-ID, vulnerability release date, and file name. We then downloaded the corresponding vulnerable and patched code, along with *diff* files, from Git[3]. Finally, we extracted the functions and statements directly related to the vulnerability fixes from the *diff* files. All vulnerability data in the CVED dataset were released between August 25, 2024, and February 24, 2025, while PailGen's baseline model, DeepSeek-Coder-V2, was released on June 17, 2024. Therefore, the CVED data is unknown to the model. Additionally, the CVED dataset strictly adheres to timeline-based splits. Specifically, vulnerabilities released between August 25, 2024, and January 10, 2025, serve as the codebase (containing 1250 samples), while data released between January 10, 2025, and February 24, 2025, constitutes the test set (containing 300 samples). This split method ensures that there are no potential data leakage issues when evaluating PailGen with the CVED dataset.

4.2 Baseline Approaches

We evaluate PailGen against five groups of baseline approaches. The first group consists of state-of-the-art (SOTA) automatic patch generation methods, including TFix [3], VRepair [7], VulRepair [19], VulMaster [73], and VQM [18]. TFix is a T5-based approach that utilizes a language model pre-trained on a natural language corpus. VRepair is a transfer learning-based vulnerability patch generation framework. It first trains a Transformer model on a large bug-fix corpus, then fine-tunes it on a vulnerability-fix dataset. VulRepair takes a vulnerable function, concatenated with its CWE ID, as input and generates patches by fine-tuning the CodeT5 [66] model. VulMaster incorporates additional information, such as vulnerability types and descriptions, and employs the Fusion in Decoder (FiD) framework [31] to address the input length limitations of the CodeT5 model. VQM is a vulnerability patch generation method that leverages Vision Transformer architecture. It employs a visual attention mechanism to enable the model to focus on vulnerable code areas within the vulnerable function.

The second group includes general-purpose pre-trained models that are widely used in various downstream programming tasks. Specifically, we compare PailGen with four well-known pre-trained models: CodeBERT [15], GraphCodeBERT [24], CodeT5 (CodeT5-Base) [66] and CodeT5+ (CodeT5+ 770M) [65]. CodeBERT is a BERT-based pre-trained language model for source code that has been widely used in various code generation tasks. GraphCodeBERT, a variant of CodeBERT, incorporates data flow information from programs during training. CodeT5 [66] is a programming language model pre-trained with a code-aware objective on large-scale code datasets. CodeT5+ [65] is an enhanced version of CodeT5 with improved capabilities. These models are fully fine-tuned, and use the same inputs and outputs as VRepair [7] and VulRepair [19]. In the third group, we compare PailGen with popular prompt tuning-based methods. Prompt tuning reshapes the training objectives of downstream tasks to align with pre-trained models, specifically leveraging Masked Language Modeling (MLM) objectives. It also modifies the model input by incorporating a natural language prompt, ensuring consistency with the pre-training phase. Prompt tuning has been shown to outperform traditional full fine-tuning in various tasks, including code summarization and code translation. To examine its strengths and limitations in vulnerability patch generation, we follow [61] and apply the optimal prompt tuning method (i.e., using a hard prompt template) to fine-tune two pre-trained models, CodeT5 and CodeT5+.

In the fourth group, to evaluate the performance of fine-tuned larger models, we select five popular code-focused large language models with parameter sizes ranging from 1B to 236B (i.e., DeepSeek-Coder-1.3B-Instruct[4],

[2]https://www.cvedetails.com/
[3]https://web.git.kernel.org/
[4]https://huggingface.co/deepseek-ai/deepseek-coder-1.3b-instruct

DeepSeek-Coder-6.7B-Instruct[5], CodeLlama-7B-Instruct[6], CodeGeeX4-All-9B[7], and DeepSeek-Coder-V2[8]) for fine-tuning. Due to resource limitations, we are unable to fully fine-tune these models. Alternatively, we apply the Low-Rank Adaptation (LoRA) tuning method. LoRA tuning is a parameter-efficient fine-tuning (PEFT) technique designed to reduce computational and storage overhead when fine-tuning large models. Its core principle is to freeze most of the pre-trained model's parameters while learning task-specific adaptations through low-rank matrices, significantly reducing the number of trainable parameters. We perform LoRA tuning using the SWIFT open-source framework [70]. The final group comprises some large models (LLMs) for code. We utilize six well-known code LLMs: CodeLlama-7B-Instruct, Magicoder-S-DS-6.7B[9], CodeGeeX4-All-9B, ChatGPT-3.5-Turbo-1106[10], DeepSeek-Coder-6.7B-Instruct, and DeepSeek-Coder-V2. Among them, ChatGPT-3.5, developed by OpenAI, is built on the GPT-3.5 architecture and can handle both natural language and programming language tasks. DeepSeek-Coder is a series of code models, each trained from scratch on 2 trillion tokens. Magicoder employs a novel approach inspired by open-source code to generate low-bias, high-quality instructional code.

4.3 Evaluation Metrics

Exact Match. Following VulMaster [73], we utilize the Exact Match (EM) metric as our primary evaluation measure. EM measures the percentage of the generated code sequence that has the same token sequence as the ground truth.

BLEU. We also employ the widely used BLEU-4 metric [51]. The BLEU metric calculates the matching degree of n-grams and includes a Brevity Penalty (BP) to address cases where the translation output is too short. In this paper, BLEU-4 is used to evaluate the token-level similarity between the generated code and the ground truth.

CodeBLEU. It is an evaluation metric specifically designed for code generation tasks [55]. It incorporates the strengths of BLEU in n-gram matching while also evaluating code syntax and semantics through AST and data flow analysis. CodeBLEU offers a comprehensive metric that assesses the alignment between the generated code and the ground truth, making it suitable for evaluating the performance of vulnerability patch generation models.

4.4 Implementation Details

Following previous studies [7, 19, 73], we focus on fixing C/C++ vulnerabilities. For the DPR retriever, we use CodeT5-Base [66] to learn dense embedding vectors from the input code sequence. CodeT5-base consists of 12 encoder layers and 12 decoder layers, with a total parameter size of 220M. The DPR retriever is trained using an in-batch negative setting with a batch size of 32. We trained the E_C and E_Q encoders for 5 epochs on our dataset, employing the Adam optimizer with a learning rate of 1e-5, linear scheduling with warm-up, and a dropout rate of 0.1.

The LLMs used in our experiment include Magicoder-S-DS-6.7B, DeepSeek-Coder-6.7B-Instruct, CodeLlama-7B-Instruct, CodeGeeX4-All-9B, ChatGPT-3.5-Turbo-1106, and DeepSeek-Coder-V2. Among them, ChatGPT-3.5-Turbo-1106 and DeepSeek-Coder-V2 are accessed via their official APIs. Magicoder-S-DS-6.7B, CodeLlama-7B-Instruct, CodeGeeX4-All-9B and DeepSeek-Coder-6.7B-Instruct are open-source models. We downloaded these models from HuggingFace[11] and ran them locally, using default hyperparameters. Specifically, for CodeLlama-7B-Instruct, the *top_p* is set to 0.95 and the temperature is set to 0.2. For Magicoder-S-DS-6.7B, the temperature is set to

[5]https://huggingface.co/deepseek-ai/deepseek-coder-6.7b-instruct
[6]https://huggingface.co/codellama/CodeLlama-7b-Instruct-hf
[7]https://huggingface.co/THUDM/codegeex4-all-9b
[8]https://chat.deepseek.com
[9]https://huggingface.co/ise-uiuc/Magicoder-S-DS-6.7B
[10]https://openai.com/blog/chatgpt
[11]https://huggingface.co/

0.5, and *num_return_sequences* is 1. The hyperparameters of DeepSeek-Coder-6.7B-Instruct and DeepSeek-Coder-V2 are $top\_p = 0.95$, and *num_return_sequences* $= 1$. Note that the parameter sizes of ChatGPT-3.5-Turbo-1106 and DeepSeek-Coder-V2 are 175 billion and 236 billion, respectively.

Since C/C++ is insensitive to indentation, we remove redundant whitespace and blank lines to ensure a fair evaluation. For each vulnerable function, we consider m predictions and generate m candidate patches per instance. If at least one of these patches exactly matches the ground truth, it indicates that the model successfully generates the correct patch. We also add two special tokens, <S2SV_StartVul> and <S2SV_EndVul>, to represent the start and end of each vulnerable code line, respectively. To prevent potential LLM API call failures due to excessively long AST or fix pattern sequences, we limit their length to 500 tokens in all experiments. Sequences exceeding this limit are truncated. For the previous approaches TFix [3], VulRepair [19], VulMaster [73] and VQM [18], we use the replication packages released by the authors and re-implement them on our dataset. Since the training code for VRepair [7] is not publicly available, we cite the results from the original paper. This is feasible because we all use the same dataset, a combination of Big-Vul and CVEfixes. To ensure a fair comparison between PailGen and the baseline models, we set the beam size to m for all baseline models. All experiments are conducted on a Linux server equipped with NVIDIA GeForce RTX 4090 GPU and Intel(R) Core(TM) i9-13900K CPU running at 4.60GHz.

5 Experimental Results

In this section, we evaluate the performance of PailGen based on the following research questions:

- **RQ1**: What is the performance of PailGen compared to state-of-the-art approaches for vulnerability patch generation?
- **RQ2**: How effective is our PailGen when applied to different Large Language Models?
- **RQ3**: What are the contributions of each component of PailGen?
- **RQ4**: How does PailGen perform across different vulnerability types (i.e., CWE-IDs)?

5.1 RQ1. Comparison with Baseline Methods

To answer this RQ, we compare PailGen with four common pre-trained models and five state-of-the-art vulnerability patch generation approaches. The evaluation metrics include EM, CodeBLEU, and BLEU, as detailed in Section 4.3. Notably, higher scores across these metrics indicate better performance.

Table 2 presents a comparison between PailGen and the baseline methods across three datasets, with the best results highlighted in bold. The experimental results demonstrate that PailGen consistently achieves the highest scores, outperforming all baseline methods. Specifically, on the Big-Vul_CVEfixes dataset, PailGen outperforms the state-of-the-art model VQM, improving the EM, CodeBLEU, and BLEU scores by 1.91%, 11.83%, and 15.21%, respectively. The root cause for this performance gap is that VQM lacks expert knowledge related to vulnerability fixes. Moreover, we observe that PailGen demonstrates significantly greater improvements in CodeBLEU and BLEU metrics compared to EM. This can be attributed to the fact that most existing vulnerability patch generation methods, such as VQM, utilize Transformer-based pre-trained models (e.g., CodeT5) to generate patches. These pre-trained models limit the length of input sequences, leading to the truncation of longer parts. In contrast, PailGen leverages LLMs to generate patches, allowing it to process longer sequences and understand the entire vulnerable code. This enables PailGen to comprehensively grasp the context of the input sequence. As a result, the patches generated by PailGen more closely resemble actual fixes.

Furthermore, VulMaster achieves competitive results in comparison to VQM. Note that the inputs of VulMaster include several fix examples corresponding to specific vulnerability types (i.e., CWE IDs), which are similar to the relevant vulnerability-fix pairs retrieved in our approach. However, these examples often differ significantly from the testing vulnerable function, making them ineffective for helping the model generate better patches.

Table 2. Comparison of PailGen with State-of-the-art Vulnerability Patch Generation Approaches and Pre-trained Models (Metrics Unit: %)

Type	Model	Big-Vul_CVEfixes			D2A			CVED		
		EM	CodeBLEU	BLEU	EM	CodeBLEU	BLEU	EM	CodeBLEU	BLEU
Full Fine Tuning	CodeBERT [15]	6.81	10.56	3.11	2.75	11.83	3.64	2.82	8.89	4.95
	GraphCodeBERT [24]	10.51	16.15	7.11	3.36	9.72	1.37	4.23	9.94	5.52
	CodeT5 [66]	10.80	38.86	25.21	5.88	32.07	15.74	4.23	29.05	14.42
	CodeT5+ [65]	14.81	38.43	15.28	7.82	35.83	13.12	7.75	33.13	12.07
Prompt Tuning	CodeT5 [66]	15.80	42.49	31.67	8.86	35.70	23.43	7.04	23.76	16.53
	CodeT5+ [65]	18.60	43.92	31.99	12.24	36.90	25.94	7.75	25.71	18.54
LoRA Tuning	DeepSeek-Coder-1.3B-Instruct	10.90	46.93	28.52	5.71	34.85	19.49	6.64	30.25	20.29
	CodeLlama-7B-Instruct	19.47	48.18	36.07	14.99	52.74	37.27	8.53	34.22	23.19
	CodeGeeX4-All-9B	19.57	45.89	35.34	14.60	45.72	32.56	12.80	41.59	27.68
	DeepSeek-Coder-6.7B-Instruct	21.60	53.84	38.17	10.78	42.43	30.99	10.43	38.56	25.07
	DeepSeek-Coder-V2	20.05	49.95	37.30	13.41	49.82	40.88	12.80	39.07	26.49
SOTA Approaches	TFix [3]	9.14	33.71	16.77	5.50	30.18	14.19	6.45	26.15	16.14
	VRepair [7]	17.61	-	-	-	-	-	-	-	-
	VulRepair [19]	18.00	42.98	34.15	10.13	32.23	19.26	8.80	31.17	18.87
	VulMaster [73]	21.02	46.55	31.30	12.90	41.81	26.58	9.65	33.00	20.73
	VQM [18]	21.32	46.37	27.69	8.03	36.28	17.49	13.40	38.78	27.82
Ours	PailGen	**23.23**	**58.20**	**42.90**	**18.74**	**58.32**	**45.61**	**17.07**	**45.54**	**27.94**

Additionally, both VRepair and VulRepair struggle to generate accurate patches, possibly because their inputs consist only of the vulnerable code and CWE ID. Relying solely on the source code and CWE ID of the vulnerable function makes it challenging for these models to learn correct fix representations, thereby limiting their performance. Additionally, we observe that PailGen significantly outperforms all four fully fine-tuned pre-trained models. Compared to the best pre-trained model, CodeT5+, PailGen improves the average EM, CodeBLEU, and BLEU scores by 9.55%, 18.22%, and 25.33%, respectively. This highlights that pre-trained models often cannot effectively fix real-world vulnerabilities. One possible reason is that these models are pre-trained on code generation datasets, which lack expert knowledge in vulnerability patch generation. As a result, even when fine-tuned on our dataset, they often fail to learn accurate fix representations.

To comprehensively evaluate the performance advantages of PailGen, we conduct comparative experiments with fine-tuned models based on prompt tuning and LoRA tuning. The results demonstrate that PailGen outperforms fine-tuned large models, such as CodeLlama-7B-Instruct, further highlighting PailGen's effectiveness in this domain. The performance advantages of PailGen stem from two key factors: 1) By extracting fine-grained structured fix patterns, PailGen captures more accurate vulnerability repair semantics; 2) Its hybrid patch retrieval module dynamically provides repair knowledge related to the target vulnerability, assisting the model in generating more precise patches. Additionally, we compare the effects of prompt tuning and full fine-tuning on the CodeT5 model. The results indicate that prompt tuning outperforms full fine-tuning in the vulnerability patch generation task, emphasizing that prompt tuning can guide the model in effectively utilizing pre-trained knowledge while avoiding over-fitting in scenarios with small sample sizes. Notably, LoRA tuning also significantly improves the performance of large models. For example, the CodeLlama-7B-Instruct model fine-tuned with LoRA shows notable improvements on the D2A dataset, surpassing the baseline model in EM, CodeBLEU,

and BLEU metrics, with increases of 14.00%, 25.70%, and 29.06%, respectively. These findings demonstrate the potential of parameter-efficient tuning methods, such as LoRA, in enhancing vulnerability patch generation. In future research, we plan to explore combining LoRA tuning with the PailGen framework to further enhance its capabilities in vulnerability patch generation.

We observe that PailGen consistently outperforms the LoRA fine-tuned DeepSeek-Coder-V2 across all three datasets, with average improvements of 4.26%, 7.74%, and 3.93% in EM, CodeBLEU, and BLEU scores, respectively. These results further demonstrate the effectiveness of our PailGen for vulnerability patch generation compared to LoRA fine-tuned LLMs. PailGen's performance advantage primarily stems from its ability to extract more targeted vulnerability fix patterns from retrieved relevant vulnerability-fix instances. Additionally, we find that the fine-tuned DeepSeek-Coder-V2 outperforms DeepSeek-Coder-6.7B-Instruct on both the D2A and CVED datasets, while performing slightly worse on the Big-Vul_CVEfixes dataset. This performance discrepancy stems from the interaction between model capacity and dataset complexity. Specifically, samples in Big-Vul_CVEfixes generally involve fewer lines of code changes compared to D2A and CVED. On average, D2A samples involve 7.9 lines of code changes, compared to 5.8 lines for samples in Big-Vul_CVEfixes. Models with smaller parameter sizes, such as DeepSeek-Coder-6.7B-Instruct, have more limited capacity, which can actually be beneficial for simpler datasets like Big-Vul_CVEfixes. The constrained capacity encourages the model to focus on learning the most critical and discriminative fix patterns, thereby improving robustness on simpler repair tasks. In contrast, the D2A and CVED datasets present more complex and diverse fix patterns, which pose greater challenges for smaller models. DeepSeek-Coder-6.7B-Instruct struggles to fully capture and generalize the intricate repair logic required, resulting in performance bottlenecks. On the other hand, given the relatively simple nature of the Big-Vul_CVEfixes dataset and the DeepSeek-Coder-6.7B-Instruct model's ability to capture its key features effectively, the larger DeepSeek-Coder-V2 model may overfit; its excessive parameters, combined with the limited data volume, can lead to degraded performance.

By comparing the experimental results of PailGen and baseline aproaches across Big-Vul_CVEfixes and D2A datasets, we find that all baseline approaches perform significantly better on the Big-Vul_CVEfixes dataset than on the D2A dataset. There are two main reasons for this performance gap. First, vulnerabilities in the wild (i.e., D2A) are generally more complex than those from official vulnerability repositories (i.e., Big-Vul_CVEfixes), often involving more potential vulnerable code statements and containing more noise in each example. Second, the inputs for these baseline methods primarily consist of vulnerable code, vulnerability types (CWE IDs), and descriptions, which are unavailable in wild scenarios. This lack of auxiliary information during model training significantly hampers models' performance in patch generation. However, PailGen achieves similar results across both datasets. The reason is that PailGen's input does not rely on vulnerability types or descriptions. Additionally, unlike pre-trained models, the LLMs used by PailGen can handle longer input sequences, making them better suited for generating complete and accurate patches. This finding highlights the robustness of PailGen.

Table 2 also shows that PailGen outperforms all baseline models on the CVED dataset, improving the EM, CodeBLEU, and BLEU scores by at least 3.67%, 3.95%, and 0.12%, respectively. This confirms that the evaluation results obtained on the Big-Vul_CVEfixes and D2A datasets are accurate and that the effectiveness evaluation is not influenced by potential data leakage issues, ensuring the validity of our findings. As shown in Table 2, most models perform slightly worse on the CVED dataset compared to the other two. This is primarily because the vulnerabilities in CVED were released after the pre-training period of most large language models, making it unlikely that these models have prior exposure to the data. Consequently, the CVED dataset offers a rigorous benchmark for evaluating the generalization ability of large language models on previously unseen vulnerabilities.
[rectangle, rounded corners, fill=blue!7, line width=0.5pt, text width=14.90cm, minimum height=2.0cm]**Answer to RQ1**: PailGen achieves the best performance, with an average EM score of 20.99% on the Big-Vul_CVEfixes and D2A datasets, among all baselines. Specifically, on the Big-Vul_CVEfixes dataset, PailGen outperforms the

Table 3. Comparison of PailGen with Baselines Using Different Large Language Models (Metrics Unit: %)

LLMs	Approach	Big-Vul_CVEfixes			D2A			CVED		
		EM	CodeBLEU	BLEU	EM	CodeBLEU	BLEU	EM	CodeBLEU	BLEU
CodeLlama-7B-Instruct	Baseline	1.62	30.35	9.73	0.99	27.04	8.21	2.37	26.41	5.36
	PailGen	**7.10**	**38.91**	**12.93**	**2.17**	**37.31**	**10.37**	**8.06**	**37.09**	**15.58**
Magicoder-S-DS-6.7B	Baseline	1.72	31.19	4.71	1.78	29.12	5.85	0.94	21.46	4.24
	PailGen	**3.85**	**38.05**	**19.24**	**6.90**	**33.21**	**19.53**	**4.16**	**29.09**	**18.53**
CodeGeeX4-All-9B	Baseline	2.64	29.47	8.73	2.56	27.82	10.47	2.37	33.44	10.57
	PailGen	**7.61**	**39.66**	**20.79**	**6.71**	**37.41**	**25.76**	**12.80**	**41.51**	**25.48**
DeepSeek-Coder-6.7B-Instruct	Baseline	4.16	35.73	22.52	4.73	35.48	28.65	3.48	29.34	15.35
	PailGen	**9.94**	**40.74**	**27.37**	**7.69**	**40.78**	**34.52**	**11.37**	**40.83**	**25.25**
ChatGPT-3.5-Turbo-1106	Baseline	5.07	32.39	12.71	6.71	32.94	16.23	7.58	36.87	18.66
	PailGen	**18.66**	**45.37**	**39.35**	**15.98**	**43.18**	**34.21**	**18.96**	**42.64**	**36.43**
DeepSeek-Coder-V2	Baseline	5.07	43.05	24.44	5.33	42.14	28.53	6.64	33.72	24.35
	PailGen	**23.23**	**58.20**	**42.90**	**18.74**	**58.32**	**45.61**	**17.07**	**45.54**	**27.94**

state-of-the-art vulnerability patch generation model VQM, improving the EM, CodeBLEU, and BLEU scores by 1.91%, 11.83%, and 15.21%, respectively.;

5.2 RQ2. The Effectiveness of Different Large Language Models

In this experiment, we evaluate the effectiveness of our PailGen across different LLMs. We utilize six code LLMs and assess their performance with (i.e., augmented prompting) and without our approach (i.e., basic prompting).

Table 3 presents our experimental results on three datasets. With our approach, DeepSeek-Coder-V2 achieves the best performance on two datasets. ChatGPT-3.5-Turbo-1106 shows suboptimal performance, with an average EM score of 17.87%. In contrast, using the baseline models without our approach results in significantly lower performance, with an EM score of 5.07% on the Big-Vul_CVEfixes dataset for both DeepSeek-Coder-V2 and ChatGPT-3.5-Turbo-1106. The experimental results clearly demonstrate the effectiveness of PailGen in improving LLM performance for vulnerability patch generation. By incorporating PailGen, the performance of all six LLMs improves substantially. DeepSeek-Coder-V2 shows an 18.16% increase in EM over the baseline, while ChatGPT-3.5-Turbo-1106 sees a 13.59% improvement on the Big-Vul_CVEfixes dataset. Additionally, we observe that open-source models with less parameters tend to underperform compared to closed-source models like DeepSeek-Coder-V2 and ChatGPT-3.5-Turbo-1106. This suggests that model size significantly influences its performance. Moreover, while DeepSeek-Coder-V2 and ChatGPT-3.5-Turbo-1106 achieve the same EM score in their baselines on the Big-Vul_CVEfixes dataset, DeepSeek-Coder-V2 outperforms ChatGPT-3.5-Turbo-1106 by 10.66% and 11.73% on the CodeBLEU and BLEU metrics, respectively. A key reason for this is that DeepSeek-Coder-V2 is trained from scratch on extensive code datasets, specifically designed for code-related tasks, allowing it to generate patches that are syntactically closer to the ground truths.

Table 3 compares PailGen's performance on the Big-Vul_CVEfixes and D2A datasets, highlighting higher Exact Match (EM) scores on Big-Vul_CVEfixes and superior BLEU scores on D2A. These differences stem from the inherent characteristics of the datasets and the distinct nature of the evaluation metrics. Specifically, the Big-Vul_CVEfixes dataset predominantly contains shorter, more localized patches than the D2A dataset. Vulnerability patches in the Big-Vul_CVEfixes dataset are generally shorter, averaging 26.7 tokens and 5.8 lines of code changes,

whereas patches in the D2A dataset are more complex, with an average of 38.4 tokens and 7.9 lines of changes. The relative simplicity of patches in Big-Vul_CVEfixes enables the model to capture and reproduce common code change patterns more effectively, increasing the likelihood of generating patches that align with reference solutions, as reflected in improved EM metrics. In contrast, BLEU evaluates n-gram similarity, allowing for partial matches, so even if a generated patch differs from the reference, it can still achieve a high BLEU score.

In summary, these findings highlight the significance of PailGen in enhancing LLMs' ability to fix vulnerabilities. By incorporating multiple sources of expert knowledge, such as relevant patches, fix patterns, and contextual information, PailGen can significantly improve LLM prompts. This enables the large models to better understand and address complex C/C++ code vulnerabilities. The consistent and significant improvements across all six models further demonstrate the robustness of PailGen.

[rectangle, rounded corners, fill=blue!7, line width=0.5pt, text width=14.9cm, minimum height=1.55cm]**Answer to RQ2**: PailGen significantly improves the performance across various LLMs. By integrating our approach, DeepSeek-Coder-V2 achieves the best results on the Big-Vul_CVEfixes and D2A datasets, with average scores of EM, CodeBLEU, and BLEU at 20.99%, 58.26%, and 44.26%, respectively.;

5.3 RQ3. The Impact of the Components of PailGen

In this experiment, we evaluate the contributions of various components of PailGen on the Big-Vul_CVEfixes dataset. Specifically, we conduct two sets of ablation experiments using different backbone models: prompt component and retrieval module designs. In each set of experiments, we remove one component at a time to assess its individual contribution.

For the prompt components, we design four variants: (i) *PailGen w/o relevant patches*: removing the component related to the relevant patches from the prompt; (ii) *PailGen w/o fix patterns*: excluding the fix patterns mined from the relevant vulnerability-fix pairs; (iii) *PailGen w/o global context*: removing the entire vulnerable function and its AST; (iv) *PailGen w/o AST*: removing the vulnerable function's AST from the prompt. For retrieval module designs, we evaluate the impact of the hybrid retrieval module on the performance of LLMs. We conduct experiments using two variants: (i) *PailGen w/o BM25 retriever*: removing BM25 and using only DPR to retrieve relevant vulnerability fixes; (ii) *PailGen w/o DPR retriever*: removing DPR and using only BM25 as the retriever. The experimental results are presented in Table 4, with the best results highlighted in bold and the worst results underlined. As shown in Table 4, we observe that the absence of any component leads to a decrease in scores across all metrics. It demonstrates that each component plays a crucial role in PailGen. Specifically, the relevant patch, fix pattern, and global context components contribute 9.03%, 4.57%, and 7.51% improvements in EM, respectively. In terms of retrieval module design, adding the BM25 and DPR retrievers results in improvements of 2.74% and 7.61% in EM, respectively.

We observe that the relevant patch is the most effective component in PailGen. After removing this component, PailGen shows a significant decline across all metrics, with EM, CodeBLEU, and BLEU scores dropping to 14.20%, 45.26%, and 29.62%, respectively. This phenomenon highlights the effectiveness of applying retrieval-augmentation techniques in vulnerability patch generation. By leveraging a hybrid retriever to select relevant vulnerability-fix pairs from the codebase and incorporating the corresponding patches (i.e., diffs) into the prompts, the performance of the PailGen is significantly improved. In most programming tasks, researchers often input entire retrieved code snippets into the model to learn their embedding representations. However, these code snippets are typically too coarse-grained to provide precise fix semantics, limiting their effectiveness in various programming tasks. PailGen avoids directly inputting the entire relevant vulnerable and fixed code into the LLM's prompt. In contrast, it extracts specific vulnerable and fixed code lines and combine them into patches. This patch format, which closely resembles the structure of the ground truth, more effectively supports PailGen in addressing vulnerabilities.

Table 4. Contributions of Different Components of PailGen (Metrics Unit: %)

Type	Variants	EM	CodeBLEU	BLEU
Full Model	PailGen	**23.23**	**58.20**	**42.90**
Prompt Components	-w/o relevant patches	14.20	45.26	29.62
	-w/o global context	15.72	44.15	34.00
	-w/o fix patterns	18.66	50.27	36.37
	-w/o AST	21.60	52.70	35.39
Retrieval Module Designs	-w/o DPR retriever	15.62	48.67	29.35
	-w/o BM25 retriever	20.49	49.50	39.05

We also observe that removing the AST sequence from the prompt leads to a slight decrease in all metrics. These results suggest that the AST contributes less to PailGen's overall performance compared to other components. The primary cause of this is the flattening of the AST into token sequences, which may lead to the loss of its hierarchical structure and node relationships. In future work, we plan to explore more advanced methods for representing the AST that better preserve its semantic structure. Additionally, we will incorporate supplementary features, such as control flow information, to further enhance the AST representation.

As shown in Table 4, removing the fix pattern component from the prompt results in a drop of 4.57%, 7.93%, and 6.53% in EM, CodeBLEU, and BLEU metrics, respectively. This emphasizes the crucial role of the fix pattern in enhancing the model's performance. While the relevant patches already provide fix semantics at the source code level, the fix patterns offer more fine-grained code change information at the AST level, enabling analysis of edits at both the statement and expression levels. Thus, the relevant patches and fix patterns complement each other in PailGen. Integrating both components allows for capturing more comprehensive fix semantics, thereby improving the accuracy of PailGen.

To further analyze the individual contribution of each prompt component in PailGen, we conduct a new set of ablation experiments. Unlike the experiments in Table 4, which remove individual modules from the complete model, this experiment evaluates each component by adding it to the basic prompt. Accordingly, we construct the following model variants: (1) *Only AST*: adds the AST of the vulnerable function to the basic prompt; (2) *Only relevant patches*: introduces components related to retrieved patches; and (3) *Only fix patterns*: incorporates fix patterns extracted from similar vulnerability-fix cases. As shown in Table 5, adding the AST, relevant patches, and fix patterns to the basic prompt improves EM performance by 5.62%, 11.26%, and 8.98%, respectively. These results confirm that each component plays a crucial role in enhancing PailGen's overall performance. Notably, relevant patches and fix patterns contribute the most significant improvements, consistent with the findings in Table 4, further validating that these components effectively guide the model toward generating correct vulnerability patches by providing structured, fine-grained semantic templates extracted from related vulnerability-fix cases.

To evaluate PailGen's sensitivity to different prompts, we design three distinct prompts, as shown in Table 6. P1 is the original prompt used in Section 5.1 and Section 5.2, P2 is a rephrased version of P1, and P3 is a simplified version of P1. This experiment explores whether PailGen can achieve consistent results across different prompts. For our experiments, we use the DeepSeek-Coder-V2 model. The results are presented in the bottom part of Table 6. Across the three prompts, we observe that PailGen's performance on the EM metric ranges from 22.67% to 23.53%, with a maximum difference of 0.86%. This demonstrates that PailGen can achieve satisfactory performance even with varying prompts.

To evaluate the impact of prompt component size on PailGen's performance, we conduct a series of experiments on the Big-Vul _CVEfixes dataset. We systematically vary the token lengths of the AST, relevant patches, and

Table 5. Decomposition of Performance Contributions by Prompt Component

Prompts of Different Variants				
Prompt Details	Basic prompting	"### Vulnerable code: *{code}* ### Task: The code contains a vulnerability. Note that <S2SV_StartVul> and <S2SV_EndVul> indicate the start and the end of vulnerable code lines. Thus the vulnerable code lines are: *{vul_line}*. Please generate a diff to fix the vulnerability."		
	Only AST	Basic prompting + "### The Abstract Syntax Tree (AST) of the code is: *{ast}*."		
	Only relevant patches	Basic prompting + "Here is an example of relevant patches: *{relevant_patch}*."		
	Only fix patterns	Basic prompting + "Here is a fix pattern extracted from the AST of relevant vulnerability-fix pair: *{fix_pattern}*."		
Metric	**Variants**	**EM**	**CodeBLEU**	**BLEU**
Results	Basic prompting	5.07	43.05	24.44
	Only AST	10.69	44.69	26.63
	Only relevant patches	16.33	48.52	33.19
	Only fix patterns	14.05	46.84	29.54

fix patterns, recording the corresponding Exact Match (EM), CodeBLEU, and BLEU metrics. First, we calculate the average length of each component in the dataset (see Table 7), with averages of 535, 39, and 4269 tokens for the AST, relevant patches, and fix patterns, respectively. Based on these averages, we set input length limits for each component, as shown in the third column of Table 7, truncating any tokens exceeding the limit. This experimental design allows for a systematic assessment of how variations in component length affect PailGen's overall performance. From these experiments, we derived the following key findings.

Firstly, regarding AST token length, as it increased from 100 to 500 tokens, all evaluation metrics steadily improved, indicating that richer structural information helps the model better understand the vulnerability context. However, when the AST length increased further to 1000 tokens, all metrics declined significantly, suggesting that excessively long ASTs may introduce redundancy or noise that interferes with the model's focus on core defects. The optimal performance threshold for AST length is therefore around 500 tokens. For relevant patch length, as the number of tokens increased from 30 to 100, the model's performance continued to improve (EM increased from 23.53% to 24.14%, CodeBLEU from 56.66% to 59.41%), demonstrating that a more complete patch context enhances the model's patch generation capability. We set the upper limit at 100 tokens, as most patches in the preprocessed dataset fall below this length. As the core component of the prompt, the fix pattern length has the most significant impact on performance. When the token length increased from 1000 to 3000, the model achieved its peak performance (EM: 24.85%), indicating that sufficient high-quality fix patterns are crucial for generating correct patches. Beyond this range (e.g., 5000 to 10,000 tokens), PailGen's performance declined, likely because overly long sequences exceed the model's effective attention range and disrupt its focus. In summary, by setting each prompt component within its optimal length range (e.g., AST: 500 tokens, fix pattern: 3000 tokens), prompt engineering can achieve an effective balance between model performance and resource efficiency.

[rectangle, rounded corners, fill=blue!7, line width=0.5pt, text width=14.9cm, minimum height=2.0cm]**Answer to RQ3**: All components are crucial for achieving optimal PailGen performance. Besides, the relevant patch is

Table 6. Impact of Varying Prompts on PailGen's Performance

		Different Prompts		
Prompt Details	P1	"### Vulnerable code: *{code}* ### The Abstract Syntax Tree (AST) of the code is: *{ast}*. ### Task: The code contains a vulnerability. Note that <S2SV_StartVul> and <S2SV_EndVul> indicate the start and the end of vulnerable code lines. Thus the vulnerable code lines are: *{vul_line}*. Please generate a diff to fix the vulnerability. Here is an example of relevant patches: *{relevant_patch}* and the fix pattern extracted from the AST of relevant vulnerability-fix pair: *{fix_pattern}*."		
	P2	"Here is a vulnerable C/C++ code: *{code}*. The code structure of the code is: *{ast}*. The <S2SV_StartVul> and <S2SV_EndVul> denote the start and end of vulnerable code statements, which are: *{vul_line}*. Please generate a diff to address the vulnerability. Below are examples of relevant patches: *{relevant_patch}*, along with the fix pattern derived from the AST of the vulnerability-fix pair: *{fix_pattern}*."		
	P3	"Vulnerable code: *{code}*. Code structure (AST): *{ast}*. <S2SV_StartVul> and <S2SV_EndVul> indicate the start and the end of vulnerable lines. Vulnerable lines: *{vul_line}*. Please generate a diff to fix the vulnerability. Relevant patches: *{relevant_patch}*. Fix patterns: *{fix_pattern}*."		
Metric		**EM**	**CodeBLEU**	**BLEU**
Results	P1	23.23	58.20	42.90
	P2	23.53	58.41	47.76
	P3	22.67	53.59	44.85

Table 7. The Effects of Prompt Component Length on PailGen Performance

Prompt Component	Average Length (#tokens)	Maximum Length	EM	CodeBLEU	BLEU
AST	535	100	23.53	56.99	42.42
		300	23.76	58.45	43.77
		500	24.04	59.10	45.72
		1000	23.23	56.51	42.53
Relevant Patches	39	30	23.53	56.66	43.50
		50	23.94	58.22	44.09
		100	24.14	59.41	44.93
Fix Patterns	4269	1000	23.66	59.06	44.59
		3000	24.85	59.77	47.09
		5000	24.14	58.46	45.42
		10,000	23.35	56.45	42.13

the most effective component in PailGen. After removing this component, PailGen shows a significant decline across all metrics, with EM, CodeBLEU, and BLEU scores dropping to 14.20%, 45.26%, and 29.62%, respectively.;

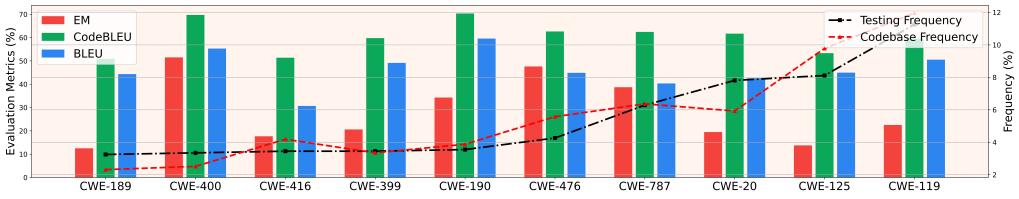


Fig. 6. The performance analysis of PailGen across different vulnerability types. The bar chart displays evaluation metrics (i.e., EM, CodeBLEU, BLEU), with the red dashed line indicating the frequency of each vulnerability type in the codebase and the black dashed line representing the testing frequency for each vulnerability type.

CWE-400: Uncontrolled Resource Consumption

```
1   bool ExprResolveBoolean(struct xkb_context *ctx, const ExprDef *expr,
2           bool *set_rtrn)
3   {
4       bool ok = false;
5       const char *ident;
6       ... /* 36 code lines are skipped */
5       case EXPR_INVERT:
6       case EXPR_NOT:
7   -           ok = ExprResolveBoolean(ctx, expr, set_rtrn);
8   +           ok = ExprResolveBoolean(ctx, expr->unary.child, set_rtrn);
9               if (ok)
10                  *set_rtrn = !*set_rtrn;
11              return ok;
12      case EXPR_ADD:
13      case EXPR_SUBTRACT:
14      case EXPR_MULTIPLY:
15      case EXPR_DIVIDE:
16      case EXPR_ASSIGN:
17      case EXPR_NEGATE:
18      case EXPR_UNARY_PLUS:
19      ... /* 9 code lines are skipped */
20      return false;
21  }
```

(a) CVE-2018-15853

CWE-189: Numeric Errors

```
1   static void tcp_illinois_info(struct sock *sk, u32 ext, struct sk_buff *skb)
2   {
3       const struct illinois *ca = inet_csk_ca(sk);
4       if (ext & (1 << (INET_DIAG_VEGASINFO - 1))) {
5           struct tcpvegas_info info = {
6               .tcpv_enabled = 1,
7               .tcpv_rttcnt = ca->cnt_rtt,
8               .tcpv_minrtt = ca->base_rtt,
9           };
10  -        u64 t = ca->sum_rtt;
11  +        if (info.tcpv_rttcnt > 0) {
12  +            u64 t = ca->sum_rtt;
13  -        do_div(t, ca->cnt_rtt);
14  -        info.tcpv_rtt = t;
15  +            do_div(t, info.tcpv_rttcnt);
16  +            info.tcpv_rtt = t;
17  +        }
18          nla_put(skb, INET_DIAG_VEGASINFO, sizeof(info), &info);
19      }
20  }
```

(b) CVE-2012-4565

Fig. 7. Two examples of fixes for the CWE-400 and CWE-189 vulnerability types.

5.4 RQ4. Results on Different Vulnerability Types

This RQ aims to assess the effectiveness of PailGen in fixing various vulnerability types (i.e., CWE IDs). By analyzing the frequency of different vulnerability types in the Big-Vul_CVEfixes dataset, we identify the 10 most common types (CWE-119, CWE-125, CWE-20, CWE-787, CWE-476, CWE-190, CWE-399, CWE-416, CWE-400, CWE-189) for evaluation.

Figure 6 presents the experimental result for each vulnerability type. We observe that PailGen achieves good performance across various vulnerabilities, generating correct patches for 25.54% of vulnerable functions affected by the top-5 common vulnerability types. On average, PailGen achieves an EM score of 26.11% across the top-10 common vulnerabilities. These results highlight the potential applicability of PailGen for fixing common vulnerabilities. Additionally, we observe that PailGen performs best on vulnerability type CWE-400 but struggles with CWE-189. The primary reason for this performance gap is that CWE-400 (i.e., Uncontrolled Resource Consumption) presents a straightforward vulnerability fix pattern, typically involving a single code line, as shown in Figure 7a. In contrast, CWE-189 (i.e., Numeric Errors) often involves improper calculation or conversion of numbers and generally spans multiple statements, making the patch generation process more complex, as

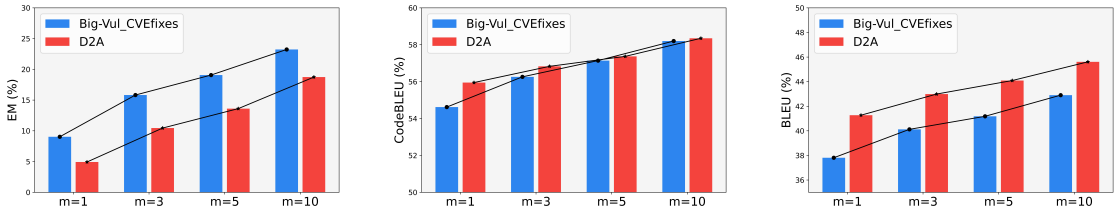


Fig. 8. The performance of PailGen with different numbers of candidate patches (i.e., different values of m).

illustrated in the example in Figure 7b. Moreover, CWE-189 vulnerabilities frequently lead to program crashes, making them particularly dangerous and requiring timely generate correct patches. Our future research will explore methods to further improve the model's ability to fix CWE-189 vulnerabilities.
[rectangle, rounded corners, fill=blue!7, line width=0.5pt, text width=14.9cm, minimum height=1.05cm]**Answer to RQ4**: PailGen performs well on the top-10 most common vulnerabilities, achieving an average EM score of 26.11%. These results demonstrate the effectiveness of PailGen for fixing common vulnerabilities.;

5.5 The Impact of Retrieval Results on PailGen's Performance

In this section, we assess how retrieval results influence PailGen's performance by addressing the following questions:

(Q1) How effective is PailGen with different numbers of candidate patches (i.e., different values of *m*)?

To ensure a fair comparison with previous studies [7, 19, 73], we assemble m prompts based on the top-k relevant vulnerability-fix pairs retrieved from the codebase, followed by the generation of m candidate patches. The m candidate patches in our experiments correspond to a beam size of m in other existing approaches. In our experiments above, m is consistently set to 10. However, manually inspecting 10 candidate patches can be time-consuming for software security developers, potentially limiting its practical application. Similar to previous approaches [8, 19], we further investigate the effectiveness of PailGen using 1, 3, and 5 candidates, which is more practical and requires less manual inspection.

Figure 8 presents the experimental results of PailGen with varying numbers of candidate patches. When analyzing the top-1 candidate patch, PailGen obtains an EM score of 9.03% on the Big-Vul_CVEfixes dataset. This indicates that PailGen can correctly fix 89 vulnerable functions out of 986 test instances by evaluating only the best solution. Moreover, increasing m significantly improves the EM score of PailGen. For instance, when m is increased from 5 to 10, the EM score rises from 19.07% to 23.23% on the Big-Vul_CVEfixes dataset. Additionally, we observe that as the value of m increases on both the Big-Vul_CVEfixes and D2A datasets, the average EM score improves significantly (rising from 6.98% to 20.99%), while scores on CodeBLEU and BLEU show only slight improvements. This indicates that even with a small m value, PailGen generates many patches that are grammatically similar to the ground truth but not fully correct. As we incorporate more relevant expert knowledge (i.e., with increasing m values), the model's ability to generate correct patches improves dramatically. This further underscores the importance of our hybrid patch retrieval and fix pattern extraction modules in effectively addressing vulnerabilities.

(Q2) Do vulnerability-fix pairs with higher similarity scores contribute more to PailGen's performance?

This experiment aims to investigate whether vulnerability-fix pairs with higher similarity scores contribute more to PailGen's performance. Table 8 presents our experimental results on the Big-Vul_CVEfixes dataset. We

Table 8. Comparison of the Effectiveness of Relevant Vulnerability-fix Cases with Different Similarities

The *k*-th Relevant Case	1	2	3	4	5	6	7	8	9	10
Semantic Similarity	0.6394	0.6344	0.6245	0.6377	0.6316	0.6184	0.6251	0.6209	0.6206	0.6207
EM	**9.03**	7.91	6.49	6.49	5.78	5.17	5.27	5.78	4.16	5.17
CodeBLEU	**54.62**	53.28	52.05	53.34	51.62	50.99	51.59	51.44	50.69	51.57
BLEU	**37.81**	37.18	35.04	37.13	34.92	35.33	34.23	34.22	34.55	35.90

observe that the model performs best when using the most similar vulnerability-fix pair, achieving an EM of 9.03%, a CodeBLEU of 54.62%, and a BLEU of 37.81%. This highlights the effectiveness of our retrieval module in identifying the most relevant fix cases. Additionally, as the similarity of the relevant fix case decreases, the EM score of the model shows a significant decline. This suggests that vulnerability-fix pairs with higher similarity scores can provide more accurate expert knowledge for large models. Thus, retrieving relevant fix cases to supply external knowledge is crucial for enhancing LLM performance. These findings further demonstrate that our hybrid retrieval module is a crucial component of PailGen.

Furthermore, to evaluate the semantic relevance between the top-*k* vulnerability-fix cases retrieved by our hybrid retrieval module and the target vulnerable code, we extract semantic embeddings for both the target vulnerable function and the vulnerable functions in the top-*k* retrieved cases using the fine-tuned CodeT5 encoder within the DPR-based semantic retrieval module. Using these embeddings, we compute the semantic correlation between each target vulnerability and its retrieved cases with cosine similarity. Cosine similarity effectively measures directional consistency in high-dimensional vector spaces and remains robust to variations in code length and surface-level differences, making it particularly suitable for evaluating semantic similarity in code. To comprehensively assess the overall retrieval quality, we compute the average semantic similarity of the top-*k* retrieved results across all test samples. As shown in the second row of Table 8, the experimental results indicate that our hybrid retrieval method achieves an average semantic similarity of 0.6394 in the top-1 retrieval task, demonstrating that the retrieved cases are highly semantically correlated with the target vulnerability and provide effective guidance for generating accurate patches.

(Q3) What is the impact of purely semantic retrieval methods on vulnerability patch generation?

To thoroughly evaluate the effectiveness of our hybrid patch retrieval module, we conduct comparative experiments against three purely semantic retrieval methods, ColBERT [34], Faiss [12] and CodeRetriever [40]. Table 9 presents the performance comparison on the Big-Vul_CVEfixes dataset. The results show that our hybrid retrieval method outperforms ColBERT, Faiss and CodeRetriever across all metrics, achieving 2.02%, 2.33% and 1.25% improvements in Exact Match (EM), respectively. This advantage stems from integrating lexical-based BM25 and semantic-based DPR retrievers. In vulnerability patch generation, BM25 efficiently retrieves fixes containing specific APIs or variable names, while DPR captures semantically equivalent but syntactically different fix patterns. This combination ensures high-confidence keyword matching while supplementing retrieval with relevant but non-overlapping fix cases, obviously enhancing the retrieval accuracy. In contrast, purely semantic methods like ColBERT and CodeRetriever struggle to match specific APIs or variable names, limiting their effectiveness in capturing fine-grained local code context required for patch generation. These findings demonstrate that our hybrid retrieval approach is superior to purely semantic methods, offering a more comprehensive solution for vulnerability patch generation.

(Q4) How do retrieval results impact final patch quality?

To assess the impact of retrieval errors on patch generation quality, we invited two senior software security experts, each with over five years of research and development experience, to conduct an empirical study. The

Table 9. The Performance Comparison of Different Retrieval Approaches

Retriever	EM	CodeBLEU	BLEU
Faiss [12]	20.90	48.74	31.18
ColBERT [34]	21.21	54.38	36.45
CodeRetriever [40]	21.98	56.73	38.39
BM25+DPR (ours)	23.23	58.20	42.90

Table 10. Experimental Results of Replacing Correctly Retrieved Patches with Irrelevant Ones

Replacement Rate	EM	CodeBLEU	BLEU
10%	22.11	58.49	43.70
30%	20.39	57.61	42.84
50%	18.76	51.16	41.43
100%	17.24	50.92	41.29

study follows a stratified random sampling method, selecting 30 vulnerability samples from the Big-Vul_CVEfixes dataset. Participants evaluated the output of PailGen's hybrid retrieval module based on pre-defined criteria, including code context relevance and vulnerability feature matching. The evaluation achieves a Cohen's kappa coefficient of 0.7945, indicating high inter-rater reliability. The average evaluation time was 128 minutes, reflecting the rigor of the process. Experimental results show an average retrieval failure rate of 41.67%, with participant A reporting 43.33% (13/30) and participant B reporting 40% (12/30). Notably, 16.67% of failed retrieval cases still resulted in correct patch generation, demonstrating PailGen's tolerance for retrieval errors.

Furthermore, we evaluate the impact of varying retrieval error rates on PailGen's performance. To do this, we introduce artificial noise into correctly retrieved samples by replacing strongly correlated patches (semantic similarity > 0.5) with irrelevant ones randomly selected from the codebase. We conduct four comparative experiments, replacing 10%, 30%, 50%, and 100% of correctly retrieved patches with unrelated ones and evaluating the corresponding impact. As shown in Table 10, the model's repair performance gradually declines as the proportion of incorrect retrieval results increases. However, even when all retrieval results are incorrect, the model maintains a repair success rate of at least 17.24%, further demonstrating the robustness of PailGen. Future research will focus on developing a more effective retrieval framework to improve the model's performance. Additionally, expanding the codebase to cover a wider range of vulnerabilities may also enhance retrieval accuracy.

We also conduct an in-depth analysis of retrieval failure cases. Through extensive observation and examination, we found that these failures mainly fall into two scenarios. First, the target vulnerability often belongs to a rare or long-tail type, for which the existing codebase lacks semantically or lexically similar precedents, rendering the retriever effectively untraceable. Second, the retriever sometimes fails to capture the deeper semantic and security characteristics of the code, largely because it relies solely on the source code of the vulnerable function, which provides insufficient contextual information for retrieving semantically relevant cases. For example, as shown in Figure 9a, the vulnerability corresponds to CWE-119 (Improper Restriction of Operations within the Bounds of a Memory Buffer). In this function, an assertion statement is present in debug mode, while in release mode it may lead to a buffer overflow. The vulnerability occurs when processing unusual tile sizes, such as YCbCr formats with subsampling. The retrieved example (shown in Figure 9b) was deemed unrelated to the target vulnerability by software security experts, leading PailGen to fail in generating the correct patch. This result indicates that PailGen's patch generation performance is influenced by the quality of the retrieval results. This retrieval failure

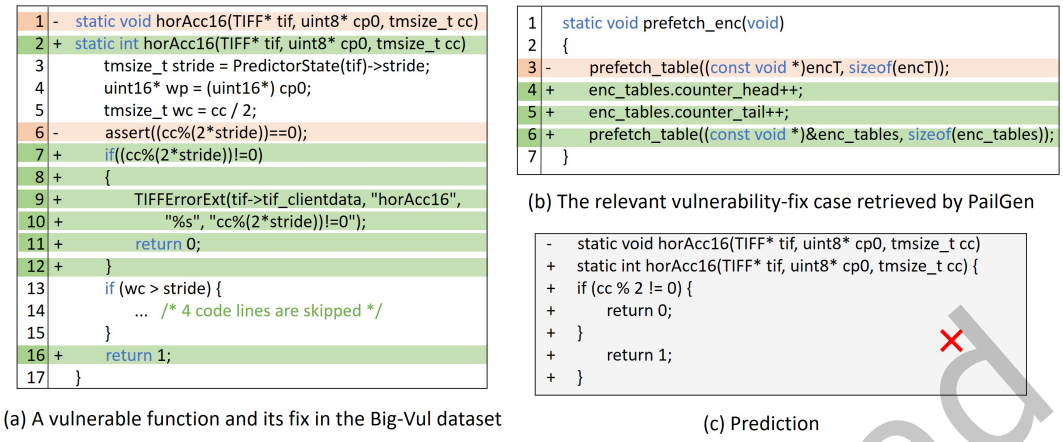


Fig. 9. A vulnerability case where PailGen failed to retrieve relevant examples and generate a correct patch.

Table 11. The Performance Comparison of Different Fix Representations

Representation	EM	CodeBLEU	BLEU
Coarse-grained	16.43	44.61	32.22
Fine-grained	22.41	55.95	46.23

case falls under the second scenario discussed above, where the retriever fails to provide sufficient contextual information to retrieve semantically relevant cases because it relies solely on the source code of the vulnerable function. The root cause of this limitation lies in the retriever's inability to leverage external knowledge (such as CWE descriptions, attack vectors, or common repair strategies) that security experts typically use for reasoning. To address this bottleneck, we plan to develop a knowledge-enhanced retrieval framework in future work. The core idea is to construct unified queries that integrate both code semantics and vulnerability patterns. To address the retrieval failure problem in the first scenario, we plan to introduce a systematic data expansion strategy in future work to enrich the codebase. This idea will be discussed in Section 7.

(Q5) Does our fix representation outperform the approach of using entire vulnerability-fix pairs?

To investigate whether our fine-grained fix representation outperforms the approach of using entire vulnerability-fix pairs, we treat the entire retrieved vulnerability-fix pairs as a coarse-grained representation, while our fix patterns mined from these pairs serve as a more fine-grained fix representation. We input these two fix representations separately into the LLM to generate vulnerability patches. Table 11 presents the experimental comparison results. We observe that our fine-grained fix representation achieves better results, with EM, CodeBLEU, and BLEU scores improving significantly by 5.98%, 11.34%, and 14.01%, respectively, over the coarse-grained representation. This improvement can be attributed to our fix pattern capturing richer contextual information than the entire vulnerability-fix pair. Unlike the original vulnerability-fix pair, our proposed fix pattern captures the code's structural characteristics, accurately locates vulnerable tokens, and analyzes code changes from multiple levels, such as the expression and statement levels. Consequently, our proposed fix representation is more effective for generating vulnerability patches than using the entire retrieved vulnerability-fix pair.

(Q6) How do the number of retrieval cases (k) and LLM prompt lengths affect PailGen's performance?

To identify the optimal balance between model performance and computational efficiency, we conduct an in-depth

Table 12. Influence of Retrieval Case Quantity (k) and Prompt Length on PailGen Performance

Prompt Length	$k=1$			$k=2$			$k=3$			$k=5$		
	EM	**CodeBLEU**	**BLEU**	**EM**	**CodeBLEU**	**BLEU**	**EM**	**CodeBLEU**	**BLEU**	**EM**	**CodeBLEU**	**BLEU**
2000	22.41	54.29	45.59	22.91	54.68	45.61	22.49	52.28	44.37	18.51	45.23	39.81
5000	24.04	56.22	47.36	24.65	56.73	47.67	23.12	54.81	46.11	19.24	45.57	40.60
8000	24.79	56.78	48.14	24.04	56.60	46.85	23.02	54.19	46.03	19.37	46.13	40.99
11,000	25.46	56.90	49.82	**25.76**	**57.14**	**50.98**	22.82	52.40	44.46	18.66	45.60	40.87
14,000	23.43	54.49	46.17	24.14	55.62	46.16	23.04	54.62	45.86	18.53	45.17	39.94
17,000	24.04	56.45	47.45	22.82	54.51	45.53	23.12	54.83	46.29	18.56	45.39	39.95
20,000	22.72	54.18	45.42	24.34	56.74	47.40	23.94	55.66	46.71	19.41	46.29	41.26

investigation of the effects of the number of retrieval examples (k) and the maximum prompt length on model performance. Here, the value of k determines how many retrieval examples are provided to the LLM during each generation, directly affecting the diversity of context information, while the prompt length limits the total information capacity of these examples. In our experiments, we set $k \in \{1, 2, 3, 5\}$ and evaluated each value with various upper limits for prompt length. The results (see Table 12) reveal a clear performance optimum: when $k = 2$ and the prompt length is approximately 11,000 tokens, the model achieves a peak Exact Match (EM) score of 25.76%. This indicates that providing two high-quality retrieval examples offers effective in-context learning signals without causing information overload. Notably, increasing k beyond 2 or extending the prompt length beyond 11,000 tokens leads to reduce performance, suggesting that excessive examples introduce redundancy or noise, while overly long contexts can divert the model's attention from critical information and degrade generation quality. Based on the system's trade-off analysis, we determine that the optimal hyperparameter configuration for PailGen is $k = 2$ retrieval examples and a prompt length of 11,000 tokens. This combination achieves optimal model performance while maintaining a balance between computational cost and inference efficiency.

6 Discussion

6.1 Differences between PailGen and Existing Approaches

The difference between PailGen and existing approaches lies in the following three aspects:

First, as one of the core contributions of this study, we develop a new automatic fix pattern mining framework that fundamentally differs from existing approaches. Existing methods typically extract fix patterns from predefined templates. However, these templates are manually defined, requiring considerable human effort and being prone to errors. Moreover, they are often restricted to specific vulnerability scenarios, limiting their effectiveness in extracting fix patterns. In contrast, we propose an automatic framework for fix pattern mining. Our approach is independent of vulnerability types, ensuring its applicability across all vulnerabilities. Specifically, we begin by constructing a structured fix representation (i.e., Minimum Edit Tree, MET) from the retrieved relevant vulnerability-fix cases. Building on this, we propose a novel fix pattern design that handles code changes at different levels (e.g., expression level and statement level). Then we develop a constrained edit path generation algorithm that converts the MET into syntax-preserving transformation paths, from which specific fix patterns are extracted. These fix patterns deconstruct the code modification process into an atomized sequence of node addition, deletion, and modification operations, while preserving syntax structure constraints and variable dependencies. In Section 5.3, we demonstrate the effectiveness of our fix pattern mining method in improving the patch generation performance of LLMs. This fine-grained pattern mining mechanism not only enhances LLM's

Table 13. The Effectiveness of DPR-BERT and DPR-CodeT5 in Vulnerability Patch Generation

Model	EM	CodeBLEU	BLEU
DPR-BERT	14.14	48.79	28.04
DPR-CodeT5	23.23	58.20	42.90

understanding of the essence of vulnerability fixes but also introduces a new technological paradigm for building interpretable automatic patch generation systems.

Second, while in-context learning is widely applied in natural language processing (NLP) and software engineering applications, its definition in this article differs from traditional methods. Traditional in-context learning typically relies on knowledge acquired during model pre-training for inference and prediction. However, pre-trained models often lack repair knowledge specific to the target vulnerable function, making it difficult to generate accurate patches. The key distinction of our approach is the development of a dynamic knowledge retrieval module that extracts highly relevant vulnerability-fix cases from the codebase in real time, providing context-enhanced information for more effective patch generation. Our contextual information goes beyond the internal code snippets of the vulnerable function, incorporating retrieved vulnerability-fix pairs and extracted fix patterns. Additionally, we integrate the structured code features by leveraging the AST within in-context learning, enabling the model to better understand code structure. By combining diverse contextual information, our approach improves large models with more precise repair knowledge, facilitating the generation of syntactically and semantically correct patches.

Third, although our hybrid retrieval module utilizes BM25 and DPR, both widely used in many NLP tasks, we are the first to apply Retrieval-Augmented Generation (RAG) technology to the field of vulnerability patch generation. It is important to note that traditional DPR methods use pre-trained BERT as a text encoder. While BERT excels in natural language representation learning, it struggles to effectively capture the syntactic structure of the code. To address this, we propose a domain adaptation-based encoder optimization strategy: using CodeT5, which is specifically designed for code understanding, to initialize both the query encoder and key encoder. We then fine-tune CodeT5 on our dataset to gain a deeper understanding of the contextual characteristics of vulnerabilities. To validate the effectiveness of this improvement, we replace the encoder in the retrieval module of PailGen with the original BERT model (denoted as DPR-BERT), while keeping the other modules unchanged. We then compare and analyze the patch generation results on the same test set. As shown in Table 13, the DPR-CodeT5 improves the EM, CodeBLEU, and BLEU metrics over DPR-BERT, with increases of 9.09%, 9.41%, and 14.86%, respectively. This result demonstrates the superiority of the improved DPR model in generating vulnerability patches.

6.2 Case Study

Figure 10 presents two fixes generated by the state-of-the-art vulnerability patch generation approaches, VulMaster and PailGen, for a CWE-476 (NULL Pointer Dereference) vulnerable function from the Big-Vul_CVEfixes dataset. A NULL pointer dereference vulnerability occurs when software dereferences a pointer that is expected to be valid but is actually NULL. This typically results in process failure unless exception handling is available and properly implemented. Even with exception handling, it is challenging to return the software to a safe operational state. In this example, the `r_pkcs7_parse_cms` function allows remote attackers to cause a denial of service (NULL pointer dereference and application crash) via a crafted PE file.

As shown in Table 2, compared to PailGen, VulMaster achieved suboptimal average results on our datasets. VulMaster [73] is a deep learning-based method that uses typical vulnerability-fix cases corresponding to the CWE

CWE-476: NULL Pointer Dereference

```
1   RCMS *r_pkcs7_parse_cms (const ut8 *buffer, ut32 length) {
2       RASN1Object *object;
3       RCMS *container;
4       if (!buffer || !length) {
5           return NULL;
6       }
7       container = R_NEW0 (RCMS);
8       if (!container) {
9           return NULL;
10      }
11      object = r_asn1_create_object (buffer, length);
12  -   if (!object || object->list.length != 2 || !object->list.objects[0] ||
13  -   object->list.objects[1]->list.length != 1) {
14
15  +   if (!object || !object->list.length || !object->list.objects[1] || !object->list.objects[0] ||   VulMaster Prediction
16  +   !object->list.objects[1]->list.length != 1) {                                                    ×
17
18  +   if (!object || object->list.length != 2 || !object->list.objects || !object->list.objects[0] ||   PailGen Prediction
19  +   !object->list.objects[1] || object->list.objects[1]->list.length != 1) {                         √
20          r_asn1_free_object (object);
21          free (container);
22          return NULL;
23      }
24      container->contentType = r_asn1_stringify_oid (object->list.objects[0]->sector, object->list.objects[0]->length);
25      r_pkcs7_parse_signeddata (&container->signedData, object->list.objects[1]->list.objects[0]);
26      r_asn1_free_object (object);
27      return container;
28  }
```

Fig. 10. An example of vulnerability fixes generated by VulMaster and our PailGen.

type of the testing vulnerable code as references for generating patches. However, in this example, we observe significant differences between these typical vulnerability-fix cases and the testing vulnerable code. Additionally, pre-trained models often fail to capture the complete contextual information of the testing vulnerable code due to input length limitations. As a result, VulMaster makes an incorrect prediction (Lines 15 and 16). In contrast, PailGen correctly generates the fixed code (Lines 18 and 19), adding checks for each pointer before dereferencing. PailGen's broader understanding of the function enables it to recognize the existing error-handling mechanism in Lines 12 and 13 (i.e., the absence of a check for potential null pointers before usage). The key reason PailGen generates accurate patches is that its hybrid retrieval module identifies fix cases with similar fix patterns from the codebase, allowing the patch generation module to extract the correct fix patterns from these cases.

Additionally, to evaluate whether the security patches generated by PailGen effectively fix vulnerabilities while preserving original functionality, we adopt the CWEVAL-BENCH datasets introduced in [53], which include curated and validated test cases. Specifically, we assess PailGen across three programming languages (i.e., C, C++, and Python) covering a total of 20 CWE types. Notably, the benchmark includes 11 C-specific tasks focused on memory-related vulnerabilities, offering a robust benchmark for secure patch generation. Following [53], we adopt two evaluation metrics (func@k and func-sec@k) adapted from the widely used pass@k metric for assessing the functionality and security of generated patches. The metric func@k measures the likelihood that at least one of the k LLM-generated patches is functionally correct, i.e., it passes all functionality test cases. The metric func-sec@k extends this by evaluating both functionality and security, indicating the likelihood that at least one generated patch passes all functionality and security test cases. In this paper, we set the value of k to 1, 5, and 10.

Table 14 presents the comparative results between PailGen and DeepSeek-Coder-V2. The findings show that PailGen consistently outperforms DeepSeek-Coder-V2 across all evaluation metrics. On average, PailGen achieves 62.12% on func@10 and 50% on func-sec@10. This means that out of every 100 samples processed by

Table 14. Functionality and Security Evaluation of PailGen-Generated Patches on CWEVAL-BENCH (%)

Model	$k = 1$		$k = 5$		$k = 10$	
	func@1	**func-sec@1**	**func@5**	**func-sec@5**	**func@10**	**func-sec@10**
DeepSeek-Coder-V2	53.03	26.77	57.16	32.52	58.08	33.84
PailGen	57.99	43.75	60.48	48.71	62.12	50.00

PailGen, approximately 62 include at least one candidate patch that successfully passes all functionality test cases. Additionally, 50 of these samples contain at least one patch that meets both functionality and security requirements. The significantly higher pass rates achieved by PailGen can be attributed to two main factors: 1) PailGen captures more precise vulnerability repair semantics by extracting fine-grained, structured fix patterns; 2) Its hybrid patch retrieval module supplies auxiliary repair knowledge relevant to the target vulnerable code, effectively guiding the model to generate syntactically and semantically correct patches. These findings further highlight the effectiveness of PailGen in enhancing LLMs' ability to generate secure patches while preserving program functionality.

6.3 Cross-Dataset Evaluation

To evaluate PailGen's performance across datasets with different distributions, we conduct two sets of experiments: (1) Group-1, using CVEfixes as the codebase and Big-Vul as the test set, and (2) Group-2, using a hybrid codebase of CVEfixes and D2A while keeping Big-Vul as the test set. These experiments are performed separately for VulRepair, VulMaster, VQM, and PailGen. Specifically, we systematically compare the four approaches across three evaluation metrics: EM, CodeBLEU, and BLEU. As shown in Table 15, PailGen demonstrates significant advantages in cross-dataset testing, achieving EM, CodeBLEU, and BLEU scores of 14.31%, 49.25%, and 34.67%, respectively, in the Group-1 setting. Compared to the suboptimal model VQM, PailGen achieves relative improvements of 1.03%, 12.39%, and 12.76%, respectively. These results highlight PailGen's strong generalization ability, which is demonstrated in two key aspects: first, it maintains superior performance on unseen vulnerability datasets, effectively transferring extracted fix patterns; second, it adapts well to different code styles and project structures, demonstrating robust contextual adaptability.

Moreover, by comparing the experimental results of Group-1 and Group-2, we find that incorporating the D2A dataset into the codebase improves PailGen's performance across all metrics (EM by 2.04%, CodeBLEU by 6.42%, and BLEU by 2.97%). This finding suggests that expanding the codebase enhances PailGen's ability to retrieve relevant vulnerability fixes, enabling it to extract more precise fix patterns. Building on this insight, future research will focus on constructing a large-scale vulnerability codebase (recommended size >100,000 samples) to fully leverage PailGen's potential in vulnerability patch generation.

To explore whether the mined fix patterns are reusable across different vulnerability types, we conduct a cross-type analysis. We group the test set by vulnerability type and, based on the 10 most common CWE categories, trace the vulnerability types of the retrieved examples used in each successfully repaired sample. We then calculate the proportion of successful repairs in which the CWE type of the retrieved example differs from that of the target vulnerability. The statistical results are shown in Table 16. We observe a significant phenomenon of pattern migration: on average, about 70% of successful repairs retrieved vulnerability-fix examples from different vulnerability types than the target vulnerability. For example, we find that the "early null check" pattern used to fix CWE-476 (NULL Pointer Dereference) vulnerabilities was successfully transferred to fix CWE-400 (Uncontrolled Resource Consumption) vulnerabilities. This finding strongly indicates that the patterns learned and utilized by PailGen are not narrowly tailored to a specific vulnerability type but rather capture more generalizable repair

Table 15. Comparison with the Baselines in the Cross-Dataset Setting

Model	Group-1			Group-2		
	EM	CodeBLEU	BLEU	EM	CodeBLEU	BLEU
VulRepair	9.77	36.01	22.08	10.78	35.32	16.67
VulMaster	13.03	38.51	22.67	15.29	39.91	23.77
VQM	13.28	36.86	21.91	12.03	36.37	15.81
PailGen	14.31	49.25	34.67	16.35	55.67	37.64

Table 16. Analysis of Fix Pattern Transferability Across Different Vulnerability Types

CWE-189	CWE-400	CWE-416	CWE-399	CWE-190	CWE-476	CWE-787	CWE-20	CWE-125	Total
81.82%	72.73%	61.54%	84.62%	78.57%	69.23%	42.31%	76.67%	42.86%	70.30%

Table 17. Performance Comparison with Baselines on the ReposVul Python Dataset

Model	EM	CodeBLEU	BLEU
VulRepair	9.86	33.87	17.93
VulMaster	12.68	34.79	19.17
VQM	14.29	36.75	23.65
PailGen	16.67	46.92	28.06

principles related to code structure integrity and security. Such transferability also explains why our method remains effective when encountering rarely seen vulnerability types in the training data, thereby mitigating the risk of overfitting to specific vulnerability categories.

Additionally, we evaluate PailGen's applicability to other programming languages, particularly dynamically typed languages like Python. We select Python code vulnerabilities from the ReposVul [64] dataset and assess PailGen's effectiveness on the collected Python subset. The results show that PailGen achieves an EM of 16.67%, a CodeBLEU of 46.92%, and a BLEU of 28.06%. Notably, PailGen performs similarly on Python as it does on C/C++ (with an EM of 18.74% for D2A), highlighting its ability to maintain high repair accuracy for dynamically typed languages. These findings suggest that PailGen generalizes well to other programming languages, driven by two key technical characteristics: (1) Language-independent hybrid retrieval: The retrieval module operates without relying on language-specific syntax rules, making it adaptable to various programming languages. (2) Unified AST-based fix pattern mining: Using the tree-sitter parser that supports multiple languages, we convert the source code into its corresponding AST and apply a unified fix pattern mining method to extract fix patterns. We also compare PailGen with three state-of-the-art baselines (i.e., VulRepair, VulMaster, and VQM) on the ReposVul (Python) dataset. As summarized in Table 17, PailGen consistently outperforms the best baseline (VQM), achieving relative improvements of 2.38% in EM, 10.17% in CodeBLEU, and 4.41% in BLEU. These results demonstrate PailGen's broad applicability and robustness across different programming languages.

6.4 Failure Case Analysis

We analyze failure cases in the Big-Vul_CVEfixes dataset and find that PailGen failures primarily occur in the following scenarios:

(1) Vulnerabilities involving incomplete security verification and improper resource lifecycle management. To assess PailGen's sensitivity to different vulnerability types, we conduct a statistical analysis of 30 vulnerability types with a sample size greater than five, covering 88.24% of the test set. The three worst-performing vulnerability types are CWE-295 (Improper Certificate Validation), CWE-772 (Missing Release of Resource After Effective Lifetime), CWE-59 (Improper Link Resolution Before File Access), all with EM scores below 12%. Among CWE-295 failure cases, 36% involve missing certificate link verification, such as failing to check intermediate CA validity in OpenSSL. CWE-772 fixes require precise operation pairing (e.g., malloc/free, open/close), but long-distance dependencies cause 44% to miss cross-function resource releases, like DMA buffers spanning three Linux driver modules. Path parsing errors in CWE-59 often arise from the interaction between symbolic links and permission handling, as seen in CVE-2019-5736, which remains unpatched due to unresolved symbolic links in `/doc/setf/fd`.

Our analysis reveals two main reasons for the failure of these vulnerability types. First, they often involve multiple function calls (e.g., signature verification, validity checks, or cross-function resource release). Since PailGen focuses on function-level fixes, it may overlook cross-function validation steps, leading to repair failures. Second, the corresponding patches are typically longer, causing the model to generate only partially correct patches. These vulnerabilities always require more extensive modifications, with an average patch length of 7.16 code lines compared to 5.15 lines for other vulnerability types. Building on these findings, we plan to integrate static analysis tools (e.g., CodeQL) to extract cross-function dependencies, enhancing the efficiency and coverage of our patch generation model. Additionally, we aim to leverage more advanced large code models to generate fixes for vulnerabilities requiring longer patches.

(2) Vulnerabilities without retrieved relevant cases. Table 10 shows a significant decline in model accuracy as the proportion of incorrect retrieval results increases from 10% to 100%, demonstrating PailGen's reliance on the quality of retrieved results. For example, in Figure 11a, PailGen retrieved a fix case for `_isdn_setup` while attempting to patch the vulnerable function `init_detection`, but the semantic similarity between them was below 0.3. Consequently, the model failed to generate the correct patch for `init_detection` due to the misleading influence of the patch "`dev->priv_flags &=~IFF_TX_SKB_SHARING;`" in `_isdn_setup`. This highlights that incorrect retrievals can mislead the model. To mitigate this, our future research will focus on developing a more efficient retrieval framework to improve the model's performance. Additionally, expanding the codebase to cover a wider range of vulnerabilities can also enhance retrieval accuracy.

6.5 Runtime Overhead

In this section, we provide a comprehensive evaluation of the runtime overheads of PailGen and baseline approaches on the Big-Vul_CVEfixes dataset. TFix and VRepair are excluded from the comparison due to their inferior performance compared to the other three baseline methods. Figure 12a presents the average runtime overheads of VulRepair, VulMaster, VQM, and PailGen. VulRepair treats vulnerability patch generation as a Neural Machine Translation (NMT) task, using only vulnerable functions as input and generating patches by fine-tuning a CodeT5 model, making it the fastest. On average, it takes only 2.22 seconds to analyze a vulnerable function in our dataset. However, as shown in Table 2, due to the lack of structured information in the code and the difficulty in capturing precise repair semantics, VulRepair produced the lowest repair quality. In contrast, VulMaster's Fusion-in-Decoder (FiD) module incorporates multiple features, including the entire vulnerable code and CWE knowledge, leading to a higher computational cost compared to VulRepair. Compared to these baseline models, PailGen incurs relatively longer runtime overheads due to the computational complexity of

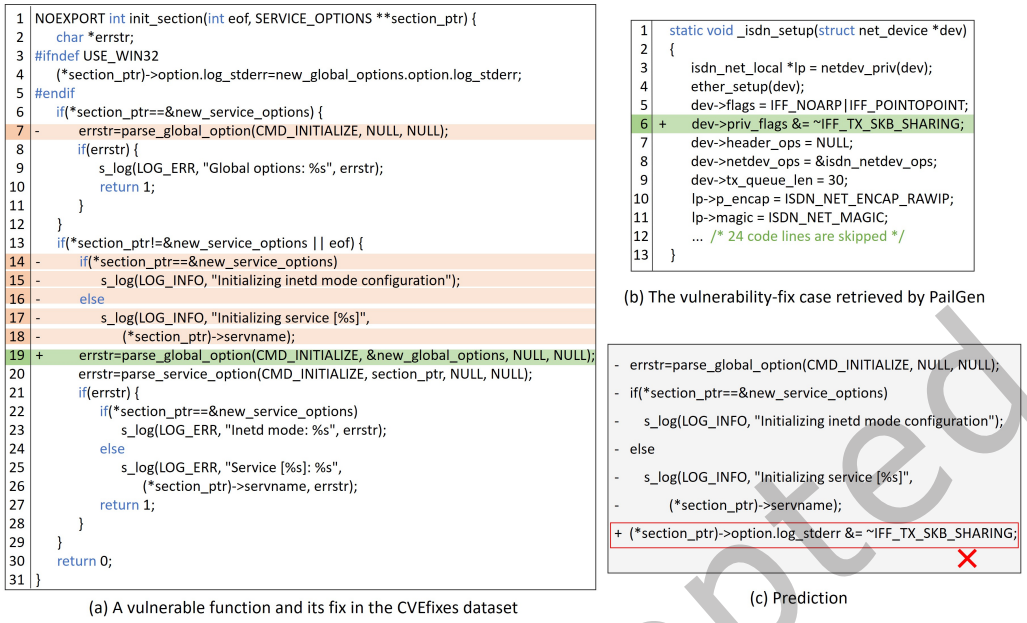
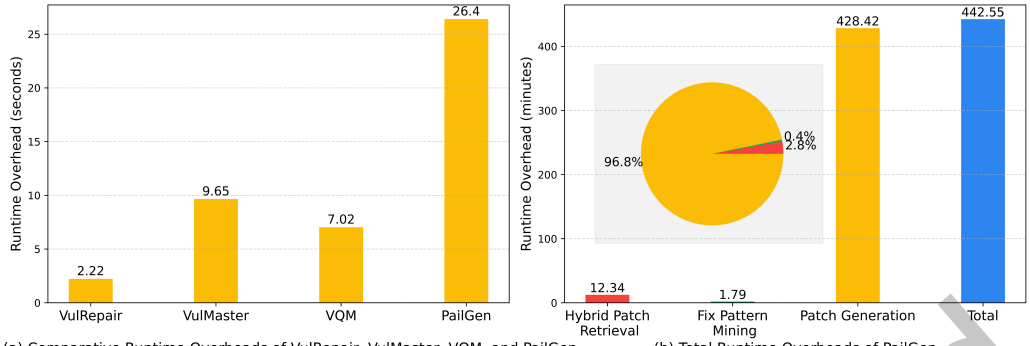


Fig. 11. An Improper Certificate Validation vulnerability (CWE-295) that PailGen fails to fix.

using LLMs for generating vulnerability patches, which is more time-consuming than leveraging a well-trained, smaller model. However, PailGen delivers the best results.

Moreover, the total processing runtime overheads of PailGen are presented in Figure 12b. Notably, after retrieving relevant fix cases for 986 testing vulnerable functions, we can extract specific fix patterns from these cases in approximately 1.79 minutes. This high efficiency demonstrates that PailGen can effectively handle large-scale software systems. We also observe that the most time-consuming phase of PailGen is patch generation, which accounts for over 96.8% of the total runtime. This indicates that PailGen's runtime is primarily influenced by the LLM inference process. Therefore, when PailGen is combined with the latest LLMs, its computational cost will be reduced. For example, DeepSeek-V3, which, despite having two to three times more parameters than DeepSeek-Coder-V2, activates less than one-tenth of the parameters per inference, thereby boosting inference speed through sparse computing.

Additionally, we analyze API call costs by empirically evaluating the feasibility of deploying PailGen with different LLMs, focusing on time and token consumption. As shown in Table 18, after prompt construction, ChatGPT-3.5 and DeepSeek-Coder-V2 require an average of 16.5 seconds and 26.4 seconds, respectively, to generate patches for each sample. Regarding token usage, the average input length per sample is approximately 4912 tokens, while the average output lengths are 383 tokens for ChatGPT-3.5 and 2893 tokens for DeepSeek-Coder-V2. Analysis of the outputs indicates that DeepSeek-Coder-V2 often produces extensive explanatory text on vulnerability principles and repair strategies before generating the actual patched code, which accounts for its longer outputs and overall generation time. Based on these token usage data and the official API billing standards, the estimated cost per sample ranges from approximately $0.0026 to $0.0056. This demonstrates that PailGen is efficient for vulnerability patch generation in terms of both time and money cost. Furthermore, although PailGen involves multiple steps, including AST parsing, relevant patch retrieval, and fix pattern mining, the entire pipeline



(a) Comparative Runtime Overheads of VulRepair, VulMaster, VQM, and PailGen (b) Total Runtime Overheads of PailGen

Fig. 12. Runtime overheads of PailGen on the Big-Vul_CVEfixes dataset.

Table 18. Time and Token Cost per Sample

Model	Time Cost	Input Tokens	Output Tokens	Price
ChatGPT-3.5-Turbo-1106	16.5s	4912	383	$0.0056
DeepSeek-Coder-V2	26.4s	4912	2893	$0.0026

is fully automated. This design allows PailGen to be easily packaged as standardized tools or plugins, seamlessly integrated into modern CI/CD pipelines, and readily applied in industrial settings.

7 Limitations of PailGen

Our experiments also show the limitations of PailGen, as it cannot generate correct patches for all types of vulnerabilities in both datasets. By analyzing the vulnerabilities that PailGen fails to fix, we conclude two possible limitations.

The first limitation is that PailGen cannot always find the most relevant vulnerability-fix cases for the testing vulnerable code. Due to the limited size of our codebase, there are often few fix cases that are closely relevant to the testing vulnerable code. As a result, even with our optimal hybrid retrieval model, the fix patterns between the retrieved code and the testing vulnerable code can diff significantly. Consequently, it may not effectively assist in generating correct patches. Figure 13 illustrates an example of Out-of-bounds Read (CWE-125) vulnerabilities, where the retrieved fix examples differ substantially from the testing vulnerable code. To fix the vulnerability shown in Figure 13a, security developers use `k5memdup0()` instead of `strdup()` to copy the realm, ensuring the correct number of bytes is allocated and preventing reads past the end of the input string. This fix pattern does not appear in the codebase. As a result, PailGen cannot find appropriate fix cases for this vulnerability and fails to resolve it. This limitation can be mitigated by adapting PailGen to a larger dataset, enabling it to retrieve more relevant vulnerability-fix examples and improve the model's performance. For rare or highly complex vulnerability patterns, the scarcity of relevant examples in the codebase hinders the model's ability to learn effective repair strategies, often leading to inaccurate patches. To mitigate this issue, our future work will pursue a systematic data expansion strategy. First, we plan to broaden the codebase by integrating large-scale code commits from diverse open-source platforms (e.g., GitHub, GitLab) and developing an automated pipeline for data cleaning and annotation, with a focus on high-quality vulnerability fixes linked to CVE IDs. Second, we will apply code augmentation techniques, such as code mutation and controllable obfuscation, to synthesize new

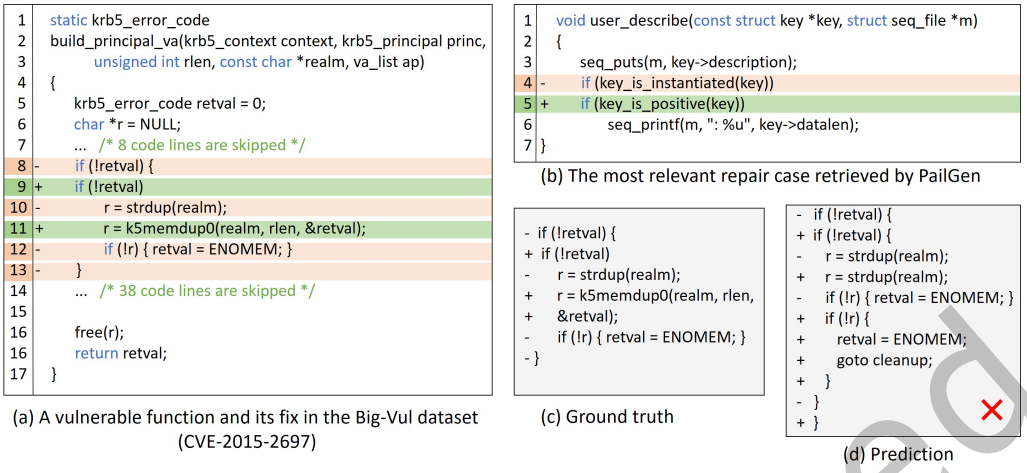


Fig. 13. An Out-of-bounds Read (CWE-125) vulnerability that PailGen fails to fix.

vulnerability samples while preserving semantic correctness, thereby enhancing model robustness. In parallel, we will explore the use of large language models to generate synthetic vulnerability-fix pairs, helping to address gaps in underrepresented patterns.

The second limitation is that the strategies used by LLMs sometimes fail to generate correct patches, even if relevant vulnerability-fix examples are retrieved. As shown in Table 3, using basic prompts (i.e., without additional knowledge) led to poor performance across all six large models, with EM scores not exceeding 10%. This highlights the limitations of LLMs in generating correct patches from basic prompts. However, by augmenting the prompts with expert knowledge, their performance improved significantly, with the highest EM score reaching 23.23%. This means PailGen can generate correct patches for approximately 23 out of 100 real-world vulnerabilities. While the accuracy of PailGen exceeds that of all baseline models, it still falls short of practical application requirements. Thus, the field of vulnerability patch generation has substantial room for improvement. We believe that this limitation can be mitigated by utilizing more advanced LLMs. To mitigate the inherent uncertainty in a single LLM's output, we generate multiple candidate patches ($k = 10$) for each vulnerable sample. This strategy broadens the search space, thereby significantly increasing the likelihood of producing the correct patch.

Additionally, PailGen focuses on fixing vulnerabilities within individual functions. Its model architecture faces inherent challenges in handling complex vulnerabilities that require cross-function analysis. This "local view" modeling approach means that PailGen may fail to recognize symptoms that manifest in one function while the root cause resides in another. Even if such issues are detected, the generated fixes may remain incomplete or invalid due to the lack of global context. Figure 14 provides an example of a Use-After-Free vulnerability (CWE-416). In the vulnerable code shown in Figure 14a, `req` is released in line 6 and subsequently used in lines 11, 14, and 19, potentially causing a memory leak. The correct fix is to move the release of `req` from line 6 to line 21. However, due to the absence of interprocedural analysis, PailGen overlooks critical program semantics. Specifically, the memory release via `mempool_free`, which involves the function call to `cifs_small_buf_release(req)` in line 6, results in incomplete semantic modeling of the vulnerable statement and failure to fix the vulnerability. In future work, we plan to integrate PailGen with static analysis tools (e.g., Joern) to address the limitations of local context. Specifically, we aim to leverage Code Property Graph (CPG) as an enhanced input representation. A CPG unifies the AST, Control Flow Graph (CFG), and Data Flow Graph (DFG) of a program into a single, comprehensive graph structure. By first constructing the CPG of the target vulnerable code using static analysis

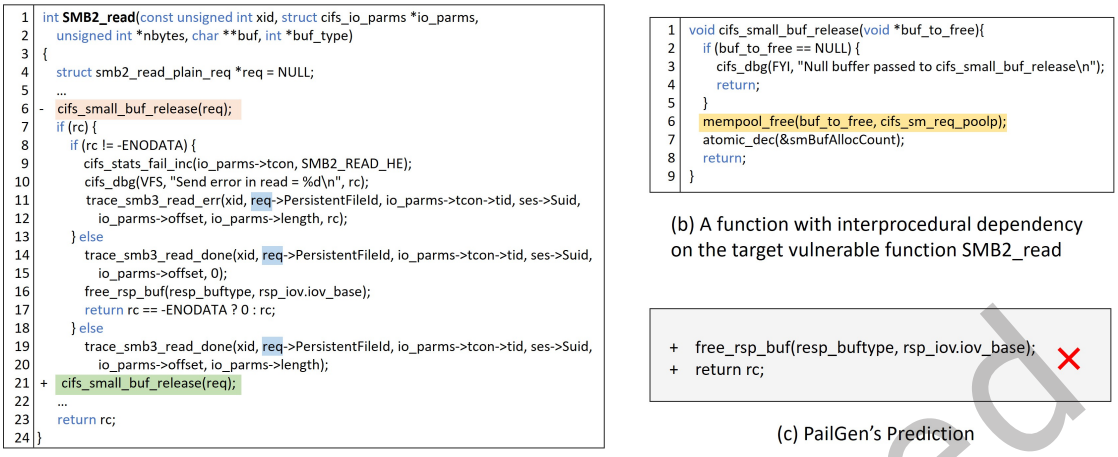


Fig. 14. A Use-After-Free vulnerability (CVE-2019-15920) that PailGen fails to fix.

tools, PailGen can capture cross-function call relationships as well as data and control dependencies, which is expected to substantially improve its repair performance.

8 Related Work

8.1 Vulnerability Patch Generation

Vulnerability patch generation is crucial for improving software security. However, vulnerability patch generation is a complex and challenging task due to the intricacies of real-world code and the diversity of potential vulnerability types. Manually fixing program vulnerabilities demands considerable time and effort. To alleviate this burden, researchers have recently proposed many automatic methods. Vulnerability patch generation aims to fix code vulnerabilities by automatically generating patches, thereby reducing the manual maintenance efforts of software developers. The vulnerability patch generation process involves generating patches for the identified vulnerabilities. Existing vulnerability patch generation techniques primarily include program analysis-based approaches, deep learning-based approaches, and LLM-based approaches.

Program Analysis-based Approaches. Program analysis-based approaches have been widely used in vulnerability patch generation, relying on pre-defined rules or patterns to identify and fix vulnerabilities in source code [28, 35]. Huang et al. [28] proposed Senx, a source code-based vulnerability patch generation method that enforces security property violations, successfully generating patches for 32 real-world vulnerabilities. Gao et al. [20] proposed a rule-based vulnerability patch generation method for smart contracts. This approach extends the repair rules of sGuard to preserve the original business logic of smart contracts while accommodating new vulnerability types. Fixminer [35] uses an iterative clustering strategy to mine relevant and actionable fix patterns. However, manually defining rules requires significant time and effort from security developers, and manually designed fix patterns may only be suitable for specific scenarios, making it challenging to cover all potential vulnerabilities.

Deep Learning-based Approaches. Among vulnerability patch generation techniques, deep learning-based approaches have recently achieved remarkable performance [7, 19, 26, 31, 73]. Many of these techniques use NMT technology to fix vulnerabilities by converting vulnerable code into a token sequence and then using deep learning models to generate patches. Chen et al. [7] introduced VRepair, which uses a word-level tokenizer and a vanilla

Transformer model to fix function-level vulnerabilities. Similar to VRepair, Chi et al. [8] proposed SeqTrans, which relies on the same components and word-level tokenizer as VRepair. Additionally, VRepair employs a copy mechanism to address out-of-vocabulary (OOV) problems. However, this mechanism limits the model's ability to generate new tokens that may not appear in existing vulnerable functions but are introduced in the patched code. To address this issue, VulRepair [19] uses the byte pair encoding (BPE) method based on the CodeT5 pre-trained model to perform subword segmentation and generate patches by fine-tuning the CodeT5 decoder. Based on the above-mentioned methods, VulMaster [73] introduced the Fusion-in-Decoder (FiD) framework [31] into the field of vulnerability patch generation. It fine-tunes the CodeT5 framework by integrating the AST structure of the vulnerable code and the corresponding CWE type to generate a patched version of the code. Harer [27] applied generative adversarial networks (GAN) to the automatic patch generation of software vulnerabilities. In addition, Michele et al. [60] explored the feasibility of using NMT techniques to generate patches. Inspired by object detection methods in computer vision, Michael et al. [18] introduced a vulnerability patch generation approach called VQM. This approach leverages attention mechanisms to focus on vulnerable code areas within a function, aiming to improve the model's ability to generate more accurate fixes. Luo et al. [44] developed CVECenter, an automatic system for vulnerability identification, assessment, and patch generation. NAVRepair [62] combines the node types of the AST with vulnerability types to provide a more precise repair context and targeted repair prompts for analyzing C/C++ vulnerabilities. Islam et al. [29] proposed a reinforcement learning-based patch generation method that integrates both semantic and syntactic reward mechanisms.

Prompting LLMs. More recently, LLMs, especially the GPT series, have demonstrated strong generalization capabilities. Consequently, these models are increasingly used in various downstream software tasks [23, 30]. By inputting the code snippet to be analyzed into the model along with specific instructions to fix the code, LLMs can automatically provide fix recommendations. The input consists of a code snippet of arbitrary length, and the model is instructed to identify potential vulnerabilities and deliver the fixed code as output. Liu et al. [41] designed four prompt templates for addressing vulnerable code and developed an iterative reasoning method to improve the efficiency of vulnerability patch generation for LLMs. Le et al. [36] examined the accuracy of LLMs, specifically ChatGPT and Bard, in identifying and addressing vulnerabilities in JavaScript programs.

Unlike the approaches mentioned above, our approach is the first to integrate retrieval-augmented fix pattern mining with in-context learning of LLMs for vulnerability patch generation. This method augments the prompts of LLMs by retrieving relevant historical vulnerability-fix pairs and then automatically constructing fix patterns. Our systematic and comprehensive evaluation demonstrates the effectiveness of our approach in fixing vulnerabilities, highlighting the significant contribution of prompt-augmented LLMs to advancing this field of research.

8.2 Large Language Models for Code

Pre-trained language models (e.g., BERT [33], T5 [54], Codex [6] and CodeT5 [66]) have significantly enhanced performance across a wide range of NLP and software development tasks. These models can be classified into encoder-only, decoder-only and encoder-decoder models based on their architectures.

Encoder-only models (e.g., BERT [33], CodeBERT [15], and GraphCodeBERT [24]) use the encoder portion of the Transformer architecture, excelling in sequence understanding by enabling bidirectional information flow and capturing rich contextual details. Decoder-only models (e.g., GPT-3 [16], GPTNeo [5], and Codex [6]) rely on the Transformer's decoder and specialize in sequence generation, producing tokens autoregressively from left to right. These models are pre-trained using unidirectional language modeling, where each token attends only to previous tokens and itself to predict the next one. Encoder-decoder models (e.g., T5 [54], BART [38], CodeT5 [66], and PLBART [1]) typically employ denoising pre-training objectives, where the input is corrupted and the decoder must reconstruct it. These models are capable of handling both sequence understanding and generation tasks.

In recent years, researchers have found that increasing the parameters and data in pre-trained models can significantly enhance the performance of LLMs, enabling them to possess advanced capabilities, such as in-context learning and progressive reasoning, which smaller models often lack. As a prominent LLM, ChatGPT [48, 50] demonstrates strong human-machine dialogue and problem-solving abilities. Following ChatGPT, many new LLMs (e.g., LLaMA [58], Auto-GPT [49], Qwen [2], PaLM [10], and ChatGLM [22]) have emerged, showcasing powerful abilities in NLP tasks. With the rapid advancement of LLMs, they have shown exceptional capabilities in programming languages, opening up new possibilities for automatic software development. Some of the most notable LLMs in this field include ChatGPT [48, 50], CodeLlama [57], DeepSeek-Coder [25], Magicoder [67], StarCoder [39], WizardCoder [45], CodeGen [46], CodeGeeX [71], CodeT5+ [65], and InCoder [17]. CodeLlama [57], are trained and fine-tuned on code data based on the Llama2 [59] model, enhancing Llama2's performance in code generation tasks. DeepSeek-Coder [25] is a series of models trained from scratch on both programming and natural language. Evaluations on programming-related benchmarks demonstrate that DeepSeek-Coder-33B-Base outperforms most existing open-source LLMs (e.g., CodeGeeX2-6B, StarCoder-Base-16B, CodeLlama-Base-34B). Various types of LLMs can be employed for vulnerability patch generation. For our research, we have selected six state-of-the-art models.

8.3 Retrieval-Augmented Generation

The general retrieval-augmented generation paradigm comprises three components: information retrieval, data augmentation, and generation models. This approach has been extensively studied in NLP, achieving state-of-the-art performance across various tasks, including question answering, question generation, and machine translation. Inspired by its success, many studies have adopted this paradigm for code intelligence tasks, such as code autocompletion [43, 68], code summarization [9, 42, 52], and code generation [52].

ReACC [43] is a retrieval-augmented code completion framework that leverages vocabulary replication and the retrieval of semantically similar code. Parvez et al. [29] improved the retrieval of relevant code or summaries from databases, using them to supplement code generation and summarization models. Similarly, Choi et al. [9] introduced a retrieval-augmented adaptive Transformer framework that enhances keyword frequency through retrieved code summaries. Du et al. [13] proposed CodeGRAG, a syntax graph retrieval-augmented code generation framework. This framework extracts data flow and control flow within code blocks, facilitating LLMs' understanding of code syntax and serving as a bridge across programming languages. RAP-Gen [63] is a retrieval-augmented framework that retrieves similar bug-fix pairs from a codebase, concatenates them with the feature vectors of the testing buggy code, and feeds them into the pre-trained model CodeT5. The CodeT5 model is fine-tuned to generate patches.

ColBERT [34] employs a late-interaction architecture that encodes queries and documents independently via BERT, followed by a lightweight yet expressive interaction mechanism to capture fine-grained semantic similarity. Faiss [12] serves as a highly efficient library for similarity search and clustering of dense vectors, capable of rapidly handling large-scale vector sets in memory, making it a foundational tool for building semantic retrieval systems. CodeRetriever [40] is a retrieval model specifically designed for code search, which uses a dual-encoder framework trained on large-scale code–natural language pairs to align code snippets and their textual descriptions into a shared representation space. While these methods excel in general-purpose semantic matching, they often struggle to accurately identify key code elements (such as vulnerable APIs or security-sensitive variables) in the context of vulnerability patch generation. To address this limitation, we propose a hybrid retrieval approach that combines DPR with BM25, effectively capturing both high-level semantic relevance and precise lexical signals.

9 Conclusion

In this paper, we propose PailGen, a novel fix pattern-aware, in-context learning-incorporated vulnerability patch generation approach. PailGen consists of three modules: a hybrid patch retrieval module that identifies vulnerability-fix pairs related to the testing vulnerable function, a fix pattern mining module that extracts specific fix patterns from the most relevant vulnerability-fix pairs, and an in-context learning-incorporated patch generation module that enhances the LLM's in-context learning and inference capabilities by incorporating multiple sources of expert knowledge, including the source code of the testing vulnerable function, its AST, relevant patches, and fix patterns. To enable effective retrieval of relevant vulnerability-fix pairs, we design a hybrid patch retriever combining BM25 and DPR. Fix pattern is essentially a fine-grained fix representation that captures the structural characteristics of the code and analyzes edits at different levels. By integrating relevant patches and fix patterns into LLM prompts, PailGen employs a powerful code language model to generate vulnerability patches. Extensive evaluation on 7465 real-world vulnerabilities demonstrates the effectiveness of PailGen.

10 Acknowledgments

This work was partly supported by the Guangdong Basic and Applied Basic Research Foundation (Grant No.2025A1515010486), the National Natural Science Foundation of China under project (Grant No.62472126), and the Shenzhen-Hong Kong Jointly Funded Project (Category A, NOSGDX20230116 091246007).

References

[1] Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified Pre-training for Program Understanding and Generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Online, 2655–2668. https://doi.org/10.18653/v1/2021.naacl-main.211

[2] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).

[3] Berkay Berabi, Jingxuan He, Veselin Raychev, and Martin Vechev. 2021. Tfix: Learning to fix coding errors with a text-to-text transformer. In *International Conference on Machine Learning*. PMLR, Virtual, 780–791.

[4] Guru Bhandari, Amara Naseer, and Leon Moonen. 2021. CVEfixes: automated collection of vulnerabilities and their fixes from open-source software. In *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '21)*. Association for Computing Machinery, New York, NY, 30–39. doi:10.1145/3475960.3475985

[5] Sid Black, Leo Gao, Phil Wang, Connor Leahy, and Stella Biderman. 2021. Gpt-neo: Large scale autoregressive language modeling with mesh-tensorflow. *If you use this software, please cite it using these metadata* 58, 2 (2021). https://doi.org/10.5281/zenodo.5297715.

[6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[7] Zimin Chen, Steve Kommrusch, and Martin Monperrus. 2022. Neural transfer learning for repairing security vulnerabilities in c code. *IEEE Transactions on Software Engineering* 49, 1 (2022), 147–165. doi:10.1109/TSE.2022.3147265

[8] Jianlei Chi, Yu Qu, Ting Liu, Qinghua Zheng, and Heng Yin. 2022. Seqtrans: automatic vulnerability fix via sequence to sequence learning. *IEEE Transactions on Software Engineering* 49, 2 (2022), 564–585. doi:10.1109/TSE.2022.3156637

[9] Yunseok Choi, Cheolwon Na, Hyojun Kim, and Jee-Hyong Lee. 2023. READSUM: Retrieval-Augmented Adaptive Transformer for Source Code Summarization. *IEEE Access* 11 (2023), 51155–51165.

[10] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.

[11] CVEdetails.com. 2024. *Vulnerabilities by type & year.* Retrieved September 29, 2025 from https://www.cvedetails.com/

[12] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The faiss library. *arXiv preprint arXiv:2401.08281* (2024).

[13] Kounianhua Du, Renting Rui, Huacan Chai, Lingyue Fu, Wei Xia, Yasheng Wang, Ruiming Tang, Yong Yu, and Weinan Zhang. 2024. CodeGRAG: Extracting Composed Syntax Graphs for Retrieval Augmented Cross-Lingual Code Generation. *arXiv preprint arXiv:2405.02355* (2024).

[14] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N Nguyen. 2020. AC/C++ code vulnerability dataset with code changes and CVE summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR '20)*. Association for Computing Machinery, New York, NY, 508–512. doi:10.1145/3379597.3387501

[15] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, Online, 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139

[16] Luciano Floridi and Massimo Chiriatti. 2020. GPT-3: Its Nature, Scope, Limits, and Consequences. *Minds and Machines* 30, 4 (2020), 681–694. https://doi.org/10.1007/s11023-020-09548-1

[17] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. In *The Eleventh International Conference on Learning Representations*. Kigali, Rwanda. https://openreview.net/forum?id=hQwb-lbM6EL

[18] Michael Fu, Van Nguyen, Chakkrit Tantithamthavorn, Dinh Phung, and Trung Le. 2024. Vision transformer inspired automated vulnerability repair. *ACM Transactions on Software Engineering and Methodology* 33, 3 (2024), 1–29. https://doi.org/10.1145/3632746

[19] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. 2022. VulRepair: a T5-based automated software vulnerability repair. In *Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering (ESEC/FSE '22)*. Association for Computing Machinery, New York, NY, 935–947. doi:10.1145/3540250.3549098

[20] Cuifeng Gao, Wenzhang Yang, Jiaming Ye, Yinxing Xue, and Jun Sun. 2024. sGuard+: Machine learning guided rule-based automated vulnerability repair on smart contracts. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 1–55.

[21] Xiang Gao, Bo Wang, Gregory J Duck, Ruyi Ji, Yingfei Xiong, and Abhik Roychoudhury. 2021. Beyond tests: Program vulnerability repair via crash constraint extraction. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 30, 2 (2021), 1–27. https://doi.org/10.1145/3418461

[22] Team GLM, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, Diego Rojas, Guanyu Feng, Hanlin Zhao, Hanyu Lai, et al. 2024. ChatGLM: A Family of Large Language Models from GLM-130B to GLM-4 All Tools. *arXiv preprint arXiv:2406.12793* (2024).

[23] Xiaodong Gu, Meng Chen, Yalan Lin, Yuhan Hu, Hongyu Zhang, Chengcheng Wan, Zhao Wei, Yong Xu, and Juhong Wang. 2024. On the effectiveness of large language models in domain-specific code generation. *ACM Transactions on Software Engineering and Methodology (TOSEM)* (2024).

[24] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, LIU Shujie, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *International Conference on Learning Representations*. Vienna, Austria. https://openreview.net/forum?id=jLoC4ez43PZ

[25] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming–The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).

[26] Rahul Gupta, Soham Pal, Aditya Kanade, and Shirish Shevade. 2017. Deepfix: Fixing common c language errors by deep learning. In *Proceedings of the aaai conference on artificial intelligence*, Vol. 31. AAAI Press, Washington, DC. doi:10.1609/aaai.v31i1.10742

[27] Jacob A Harer, Onur Ozdemir, Tomo Lazovich, Christopher P Reale, Rebecca L Russell, Louis Y Kim, and Peter Chin. 2018. Learning to repair software vulnerabilities with generative adversarial networks. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems (NIPS'18)*. Curran Associates Inc., Red Hook, NY, USA, 7944–7954.

[28] Zhen Huang, David Lie, Gang Tan, and Trent Jaeger. 2019. Using safety properties to generate vulnerability patches. In *2019 IEEE symposium on security and privacy (SP)*. IEEE, SAN FRANCISCO, CA, 539–554. doi:10.1109/SP.2019.00071

[29] Nafis Tanveer Islam, Mohammad Bahrami Karkevandi, and Peyman Najafirad. 2024. Code security vulnerability repair using reinforcement learning with large language models. *arXiv preprint arXiv:2401.07031* (2024).

[30] Nafis Tanveer Islam, Joseph Khoury, Andrew Seong, Gonzalo De La Torre Parra, Elias Bou-Harb, and Peyman Najafirad. 2024. LLM-Powered Code Vulnerability Repair with Reinforcement Learning and Semantic Reward. *arXiv preprint arXiv:2401.03374* (2024).

[31] Gautier Izacard and Édouard Grave. 2021. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. Association for Computational Linguistics, Online, 874–880. doi:10.18653/v1/2021.eacl-main.74

[32] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen Tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics (ACL), Online, 6769–6781. doi:10.18653/v1/2020.emnlp-main.550

[33] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of naacL-HLT*, Vol. 1. Association for Computational Linguistics, Minneapolis, Minnesota, 2. https://doi.org/10.18653/v1/N19-1423

[34] Omar Khattab and Matei Zaharia. 2020. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 39–48.

[35] Anil Koyuncu, Kui Liu, Tegawendé F Bissyandé, Dongsun Kim, Jacques Klein, Martin Monperrus, and Yves Le Traon. 2020. Fixminer: Mining relevant fix patterns for automated program repair. *Empirical Software Engineering* 25 (2020), 1980–2024. doi:10.1007/s10664-

019-09780-z

[36] Tan Khang Le, Saba Alimadadi, and Steven Y Ko. 2024. A Study of Vulnerability Repair in JavaScript Programs with Large Language Models. In *Companion Proceedings of the ACM on Web Conference 2024*. Association for Computing Machinery, New York, NY, 666–669. https://doi.org/10.1145/3589335.3651463

[37] Junhee Lee, Seongjoon Hong, and Hakjoo Oh. 2018. Memfix: static analysis-based repair of memory deallocation errors for c. In *Proceedings of the 2018 26th ACM Joint meeting on European software engineering conference and symposium on the foundations of software engineering*. Association for Computing Machinery, New York, NY, 95–106. https://doi.org/10.1145/3236024.3236079

[38] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 7871–7880. doi:10.18653/v1/2020.acl-main.703

[39] R Li, LB Allal, Y Zi, N Muennighoff, D Kocetkov, C Mou, M Marone, C Akiki, J Li, J Chim, et al. 2023. StarCoder: May the Source be With You! *Transactions on machine learning research* (2023).

[40] Xiaonan Li, Yeyun Gong, Yelong Shen, Xipeng Qiu, Hang Zhang, Bolun Yao, Weizhen Qi, Daxin Jiang, Weizhu Chen, and Nan Duan. 2022. Coderetriever: A large scale contrastive pre-training method for code search. In *Proceedings of the 2022 conference on empirical methods in natural language processing*. 2898–2910.

[41] Peng Liu, He Wang, Chen Zheng, and Yuqing Zhang. 2024. Prompt fix: Vulnerability automatic repair technology based on prompt engineering. In *2024 International Conference on Computing, Networking and Communications (ICNC)*. IEEE, Big Island, HI, USA, 116–120. https://doi.ieeecomputersociety.org/10.1109/ICNC59896.2024.10556123

[42] Shangqing LIU, Yu CHEN, Xiaofei XIE, Jingkai SIOW, and Yang LIU. 2021. Retrieval-augmented generation for code summarization via hybrid GNN.(2021). In *Proceedings of the Ninth International Conference on Learning Representations: ICLR*. Vienna, Austria, 4–8.

[43] Shuai Lu, Nan Duan, Hojae Han, Daya Guo, Seung-won Hwang, and Alexey Svyatkovskiy. 2022. ReACC: A Retrieval-Augmented Code Completion Framework. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Dublin, Ireland, 6227–6240. https://doi.org/10.18653/v1/2022.acl-long.431

[44] Jing Luo, Heyuan Shi, Yongchao Zhang, Runzhe Wang, Yuheng Shen, Yuao Chen, Xiaohai Shi, Rongkai Liu, Chao Hu, and Yu Jiang. 2024. Cvecenter: Industry practice of automated vulnerability management for linux distribution community. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 329–339.

[45] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. *arXiv preprint arXiv:2306.08568* (2023).

[46] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. *ICLR* (2023).

[47] Yu Nong, Mohammed Aldeen, Long Cheng, Hongxin Hu, Feng Chen, and Haipeng Cai. 2024. Chain-of-thought prompting of large language models for discovering and fixing software vulnerabilities. *arXiv preprint arXiv:2402.17230* (2024).

[48] OpenAI. 2022. *Introducing ChatGPT*. Retrieved September 8, 2024 from https://openai.com/blog/chatgpt

[49] OpenAI. 2023. *Auto-GPT*. Retrieved September 8, 2024 from https://news.agpt.co/

[50] OpenAI. 2023. *GPT-4*. Retrieved September 8, 2024 from https://openai.com/research/gpt-4

[51] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics (ACL '02)*. Association for Computational Linguistics, Stroudsburg, PA, 311–318.

[52] Md Rizwan Parvez, Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Retrieval Augmented Code Generation and Summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2021*. Association for Computational Linguistics, Punta Cana, Dominican Republic, 2719–2734. https://doi.org/10.18653/v1/2021.findings-emnlp.232

[53] Jinjun Peng, Leyi Cui, Kele Huang, Junfeng Yang, and Baishakhi Ray. 2025. CWEval: Outcome-driven Evaluation on Functionality and Security of LLM Code Generation. *arXiv preprint arXiv:2501.08200* (2025).

[54] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.

[55] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020).

[56] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389. http://dx.doi.org/10.1561/1500000019

[57] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[58] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and

Efficient Foundation Language Models. *ArXiv* abs/2302.13971 (2023). https://api.semanticscholar.org/CorpusID:257219404

[59] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[60] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An empirical study on learning bug-fixing patches in the wild via neural machine translation. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 28, 4 (2019), 1–29. https://doi.org/10.1145/3340544

[61] Chaozheng Wang, Yuanhang Yang, Cuiyun Gao, Yun Peng, Hongyu Zhang, and Michael R Lyu. 2022. No more fine-tuning? an experimental evaluation of prompt tuning in code intelligence. In *Proceedings of the 30th ACM joint European software engineering conference and symposium on the foundations of software engineering*. 382–394.

[62] Ruoke Wang, Zongjie Li, Chaozheng Wang, Yang Xiao, and Cuiyun Gao. 2024. NAVRepair: Node-type Aware C/C++ Code Vulnerability Repair. *arXiv preprint arXiv:2405.04994* (2024).

[63] Weishi Wang, Yue Wang, Shafiq Joty, and Steven CH Hoi. 2023. Rap-gen: Retrieval-augmented patch generation with codet5 for automatic program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. Association for Computing Machinery, New York, NY, 146–158. doi:10.1145/3611643.3616256

[64] Xinchen Wang, Ruida Hu, Cuiyun Gao, Xin-Cheng Wen, Yujia Chen, and Qing Liao. 2024. ReposVul: A Repository-Level High-Quality Vulnerability Dataset. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion '24)*. Association for Computing Machinery, New York, NY, 472–483. doi:10.1145/3639478.3647634

[65] Yue Wang, Hung Le, Akhilesh Gotmare, Nghi Bui, Junnan Li, and Steven Hoi. 2023. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 1069–1088. doi:10.18653/v1/2023.emnlp-main.68

[66] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Punta Cana, Dominican Republic, 8696–8708. doi:10.18653/v1/2021.emnlp-main.685

[67] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2023. Magicoder: Source Code Is All You Need. *arXiv preprint arXiv:2312.02120* (2023).

[68] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Singapore, 2471–2484. https://doi.org/10.18653/v1/2023.emnlp-main.151

[69] Quanjun Zhang, Chunrong Fang, Tongke Zhang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2023. Gamma: Revisiting template-based automated program repair via mask prediction. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Kirchberg, Luxembourg, 535–547. doi:10.1109/ASE56229.2023.00063

[70] Yuze Zhao, Jintao Huang, Jinghan Hu, Xingjun Wang, Yunlin Mao, Daoze Zhang, Zeyinzi Jiang, Zhikai Wu, Baole Ai, Ang Wang, et al. 2024. Swift: a scalable lightweight infrastructure for fine-tuning. *arXiv preprint arXiv:2408.05517* (2024).

[71] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. 2023. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*. Association for Computing Machinery, New York, NY, 5673–5684. doi:10.1145/3580305.3599790

[72] Yunhui Zheng, Saurabh Pujar, Burn Lewis, Luca Buratti, Edward Epstein, Bo Yang, Jim Laredo, Alessandro Morari, and Zhong Su. 2021. D2a: A dataset built for ai-based vulnerability detection methods using differential analysis. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, Madrid, Spain, 111–120. doi:10.1109/ICSE-SEIP52600.2021.00020

[73] Xin Zhou, Kisub Kim, Bowen Xu, DongGyun Han, and David Lo. 2024. Out of Sight, Out of Mind: Better Automatic Vulnerability Repair by Broadening Input Ranges and Sources. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. Association for Computing Machinery, New York, NY, 1–13. doi:10.1145/3597503.3639222

Received 20 November 2024; revised 25 October 2025; accepted 6 January 2026